



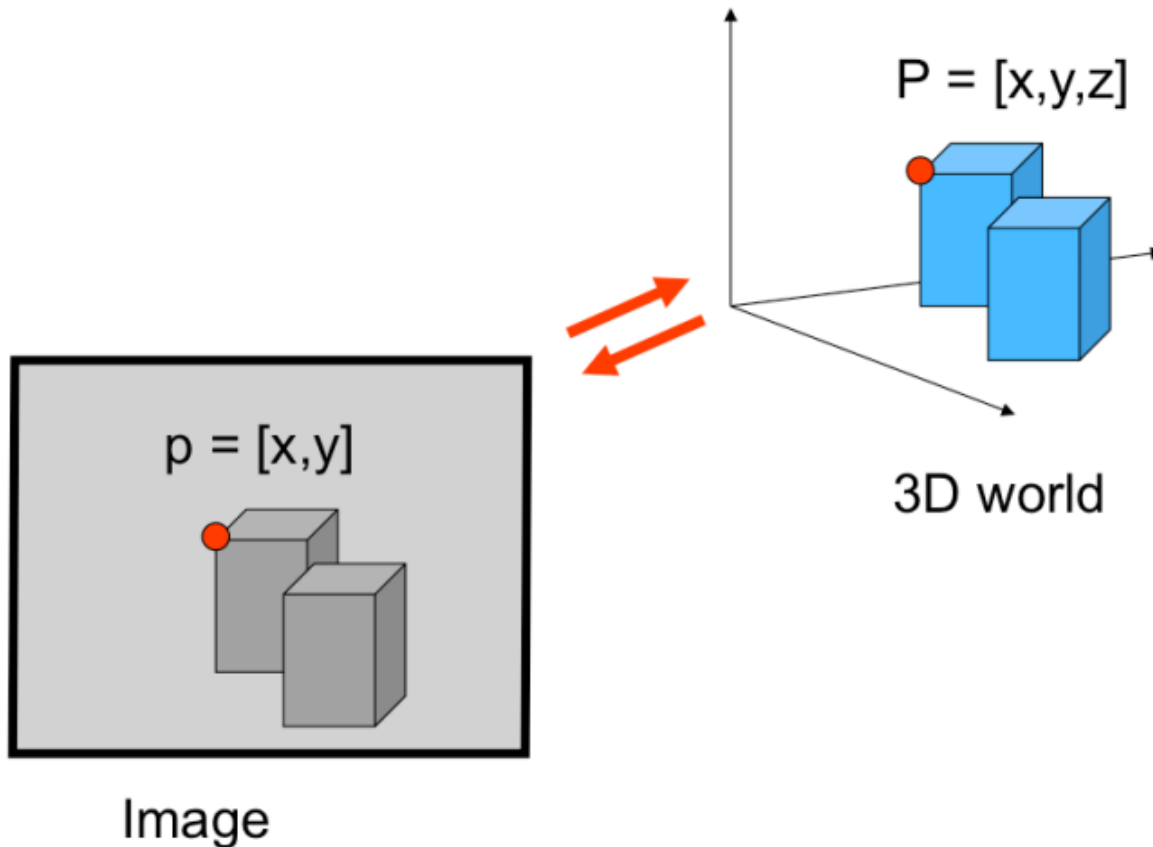
UNIVERSITÀ DI PARMA

Features Extraction



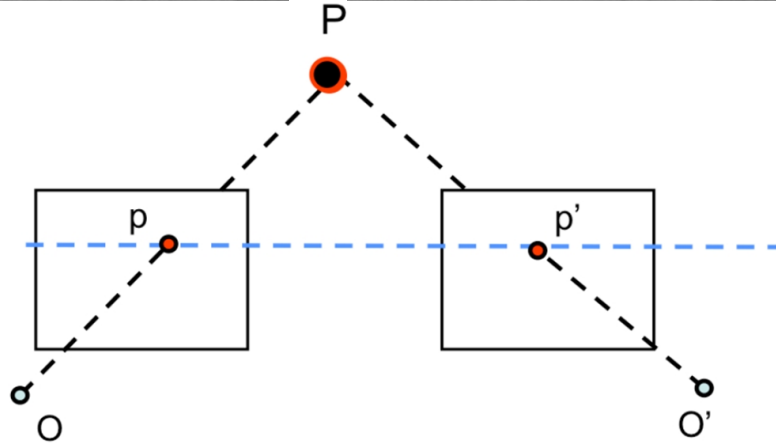
- What is a feature?
- Keypoints ideal characteristics
- Harris Corners
- Scale Invariant Detectors

- [FP] D. A. Forsyth and J. Ponce. **Computer Vision: A Modern Approach** (2nd Edition). Prentice Hall, 2011.
- **CS231A · Computer Vision: from 3D reconstruction to recognition**, Prof. Silvio Savarese – Stanford University
- **CS131 · Computer Vision: Foundations and Applications**, Prof. Fei-Fei Li – Stanford University
- **Elementi di Analisi per Visione Artificiale**
 - Paolo Medici <http://www.ce.unipr.it/people/medici>



- We found a geometrical relation between world and image

Small Recap



Beyond Stereo Vision

UNIVERSITÀ
DI PARMA



Beyond Stereo Vision



by [Diva Sian](#)

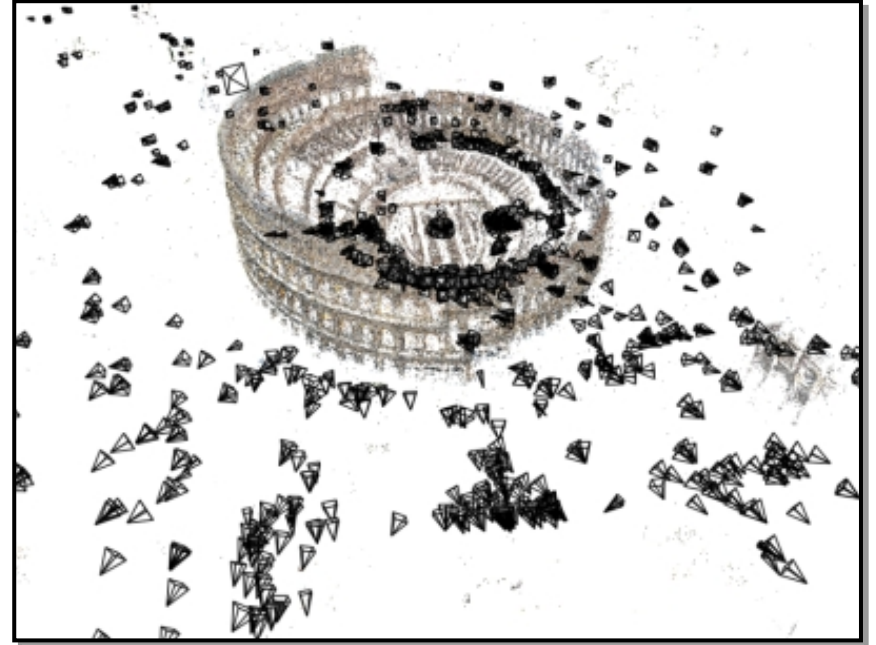
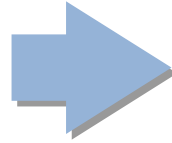


Beyond Stereo Vision

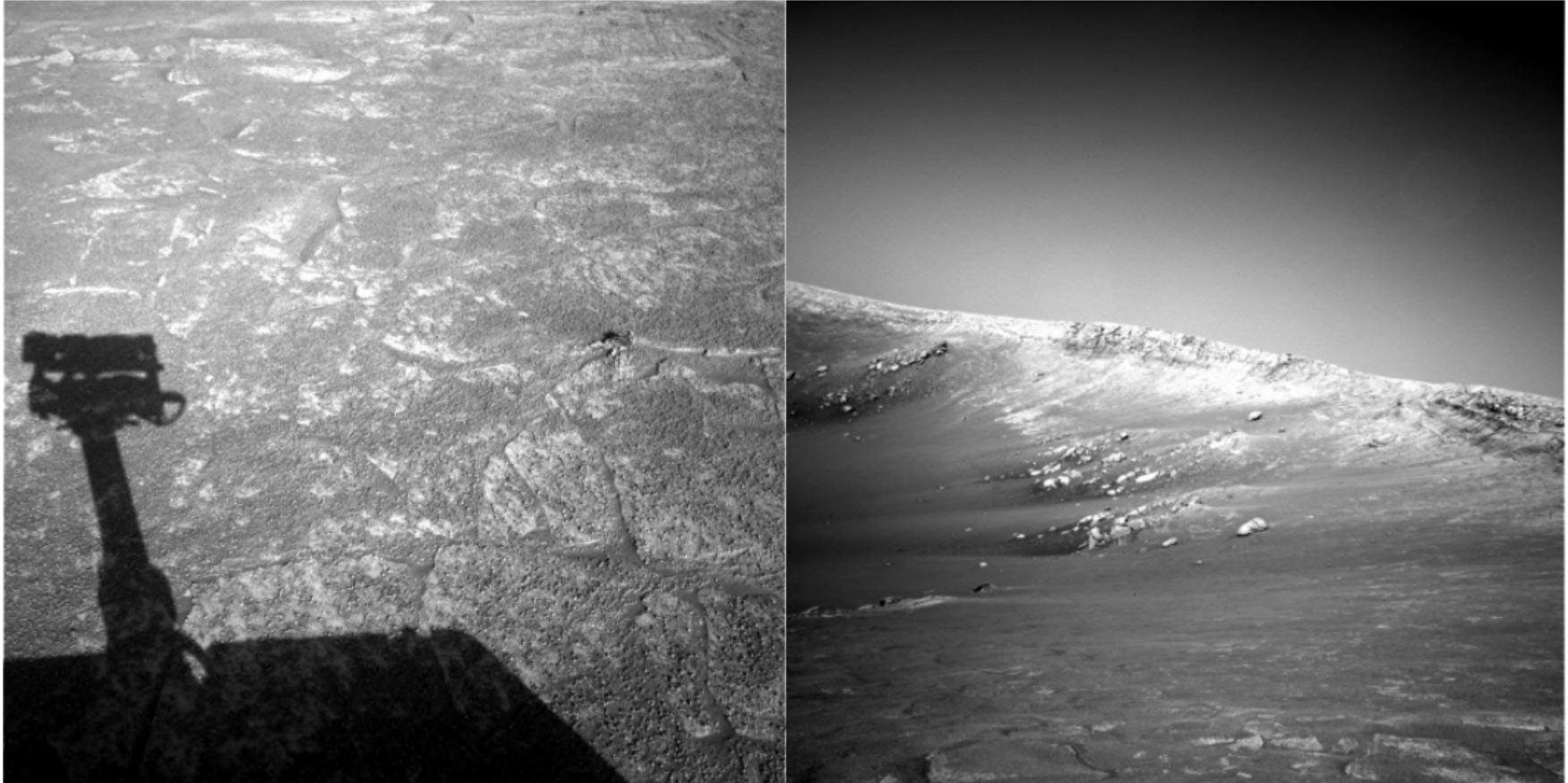
UNIVERSITÀ
DI PARMA



Beyond Stereo Vision

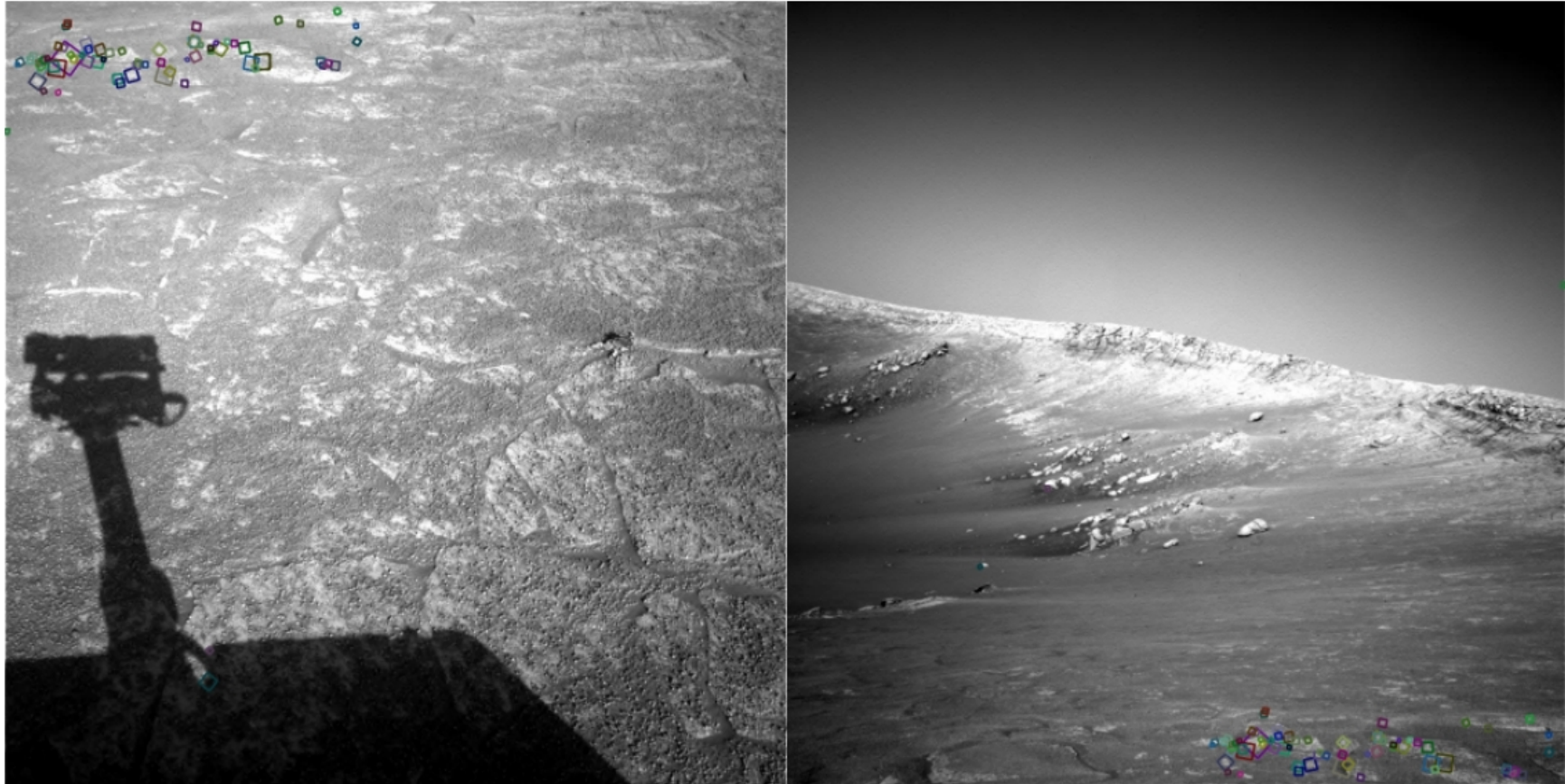


Beyond Stereo Vision



NASA Mars Rover images

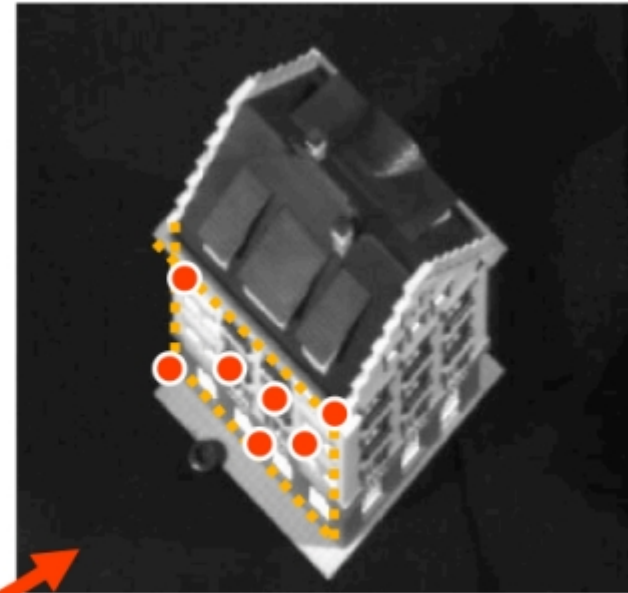
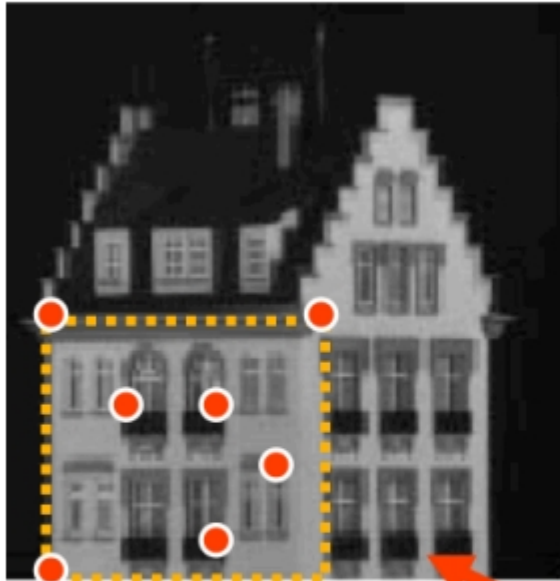
Beyond Stereo Vision



NASA Mars Rover images with SIFT feature matches

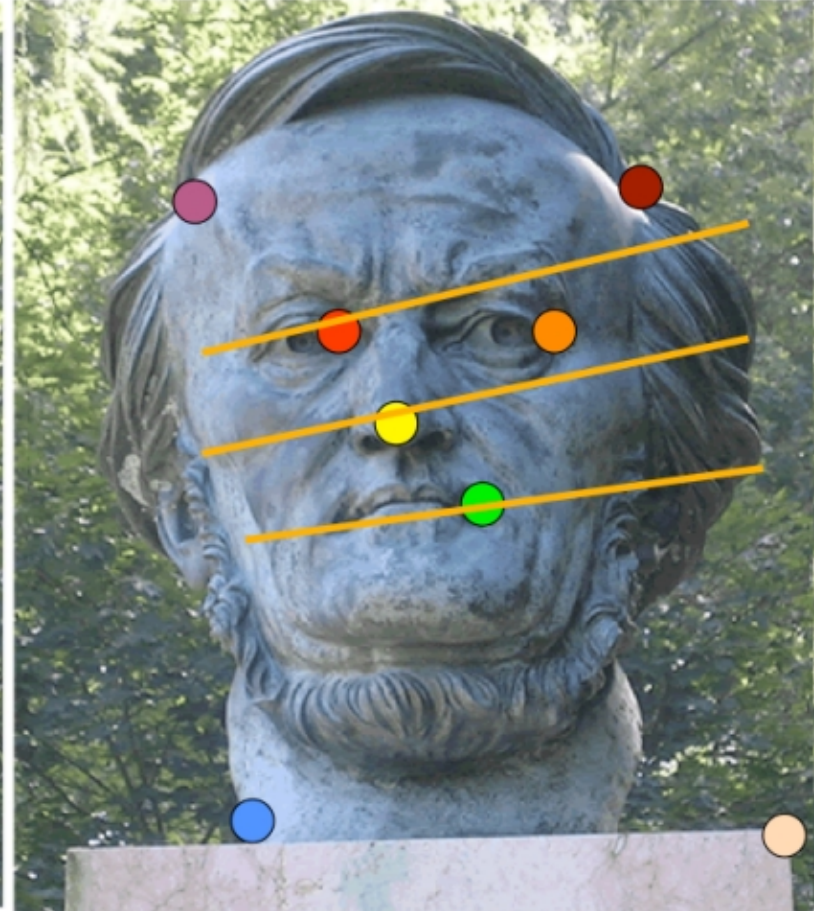
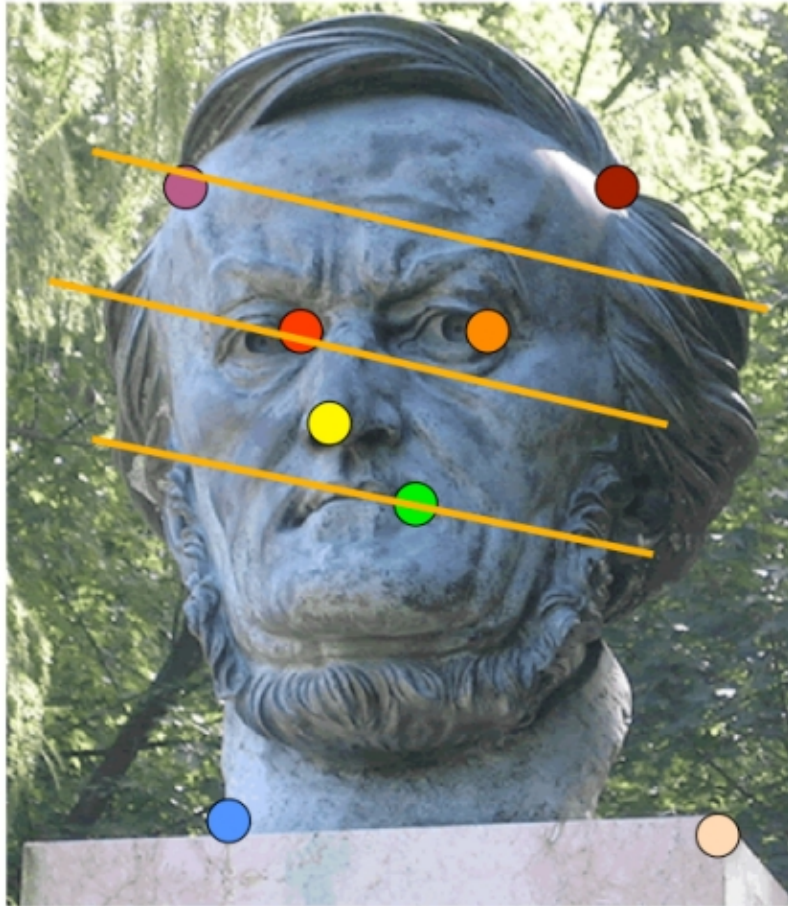
- During this lesson and the following one we are going to learn:
 - What are “features” and their properties
 - How to extract “features” from images
 - How to match them among all images

- Potential outcomes are:
 - Fit or estimate models that describe an image or a portion of it
 - Match or index images
 - Detect objects or actions from images
 - “Combine” different images

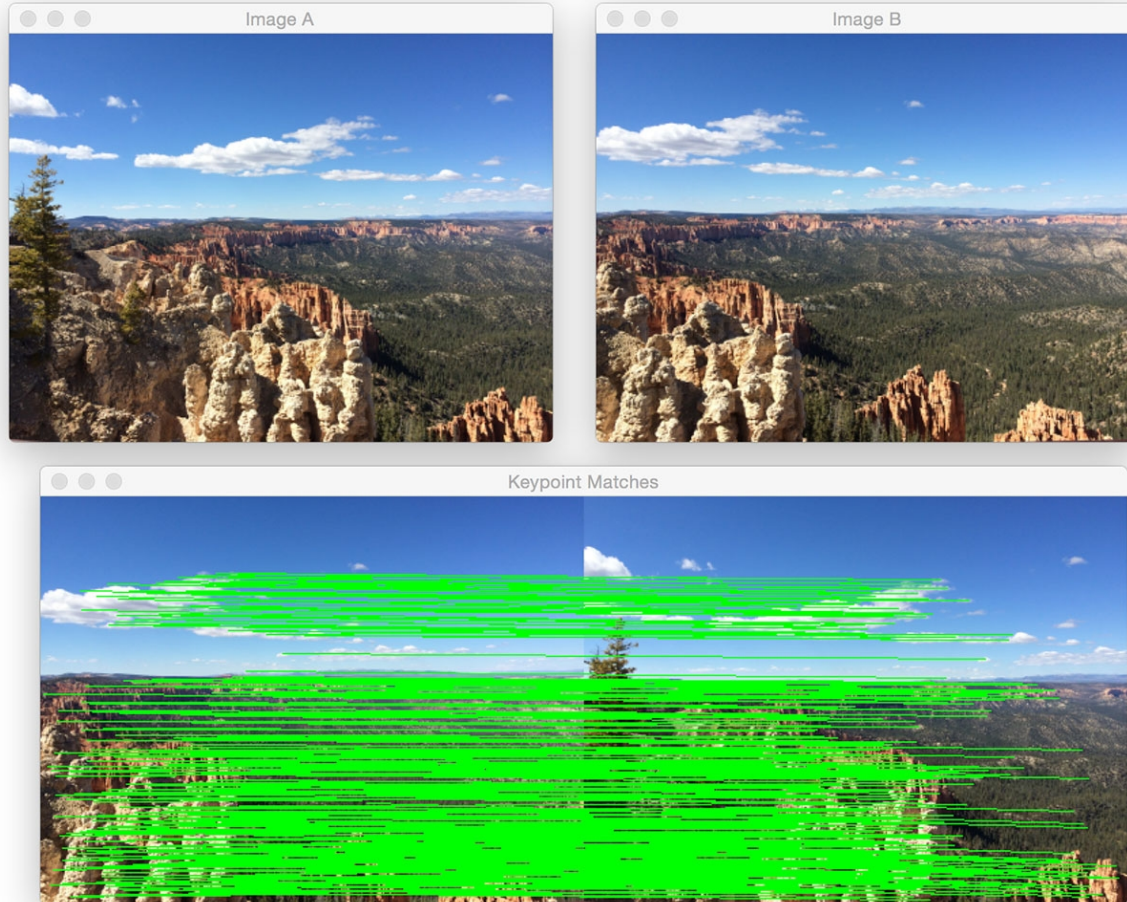


H

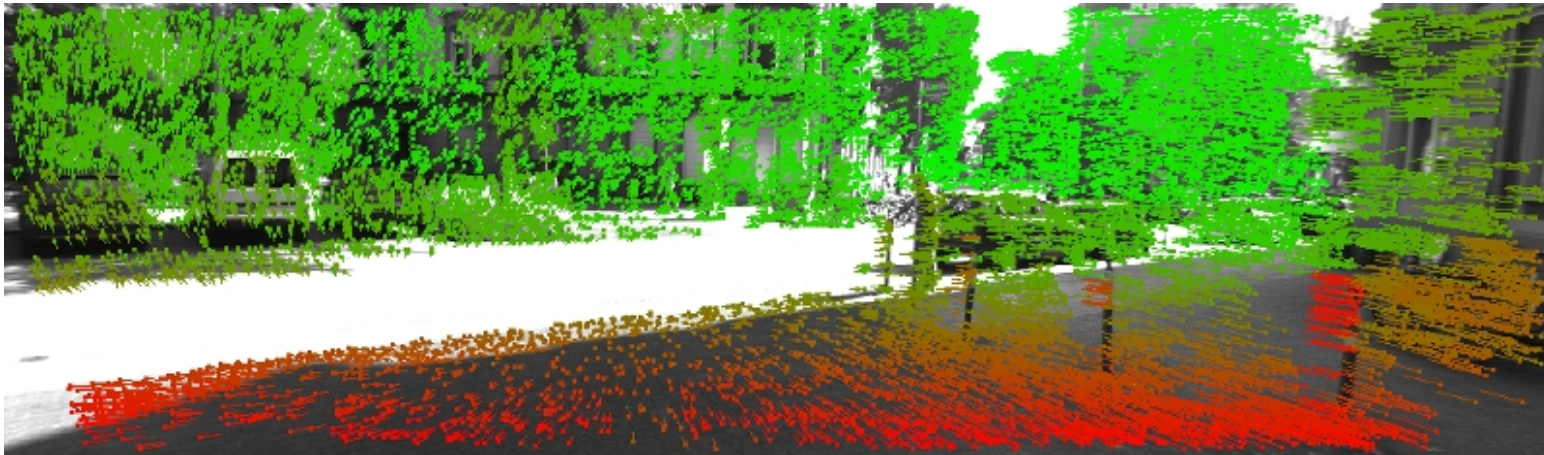
Applications



Applications



Applications



Applications



Applications



- Theoretically we could use stereo vision techniques
 - Too much expensive!
 - Moreover, we do not have to obtain a full 3D reconstruction
- Just look at the following case
 - Joining of different images...

Example

- How to combine following images?



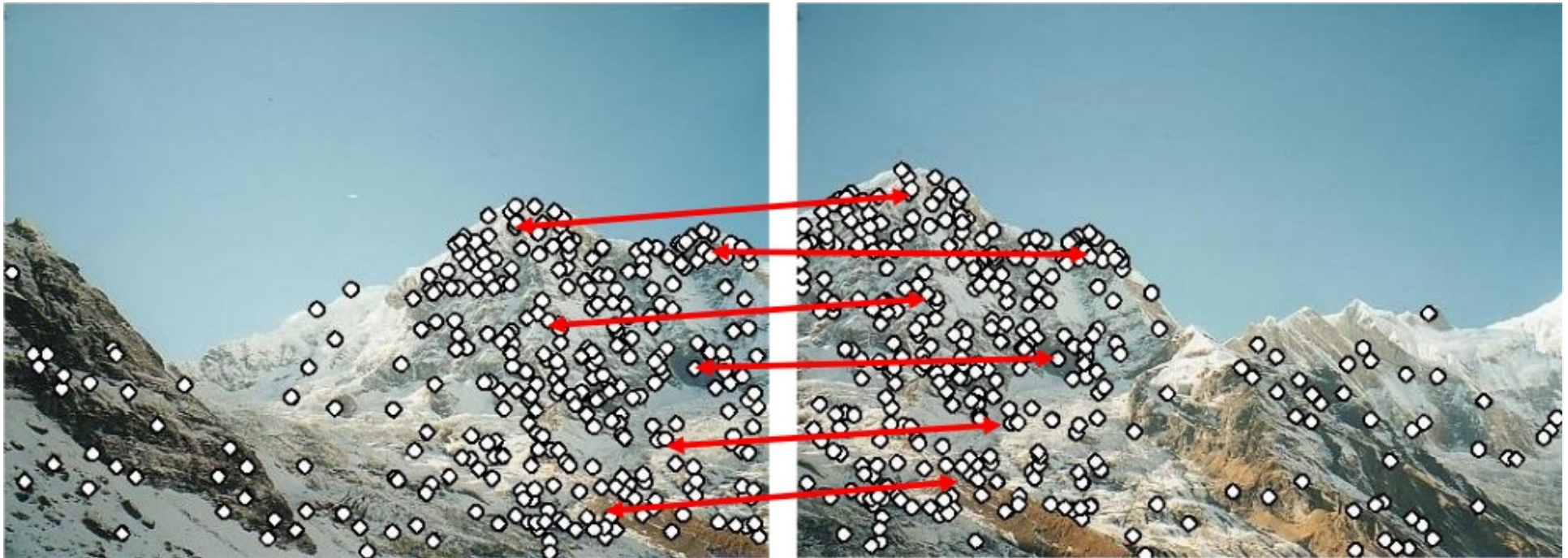
Example

- 1. select “specific” points



Example

- 2. find potential correspondences



- 3. compute alignment



- Select specific points $\rightarrow$ keypoints or features $\rightarrow$ **features extraction**
- Find potential correspondences $\rightarrow$ **features matching**
- Application dependent step
 - Matching
 - Indexing
 - Detection
 - Image alignment

- For Stereo Vision
 - Window match $\rightarrow$ SAD
- In a more general situation
 - Different size
 - Different orientation
- A complete match is impracticable
 - Only keypoints $\rightarrow$ patches

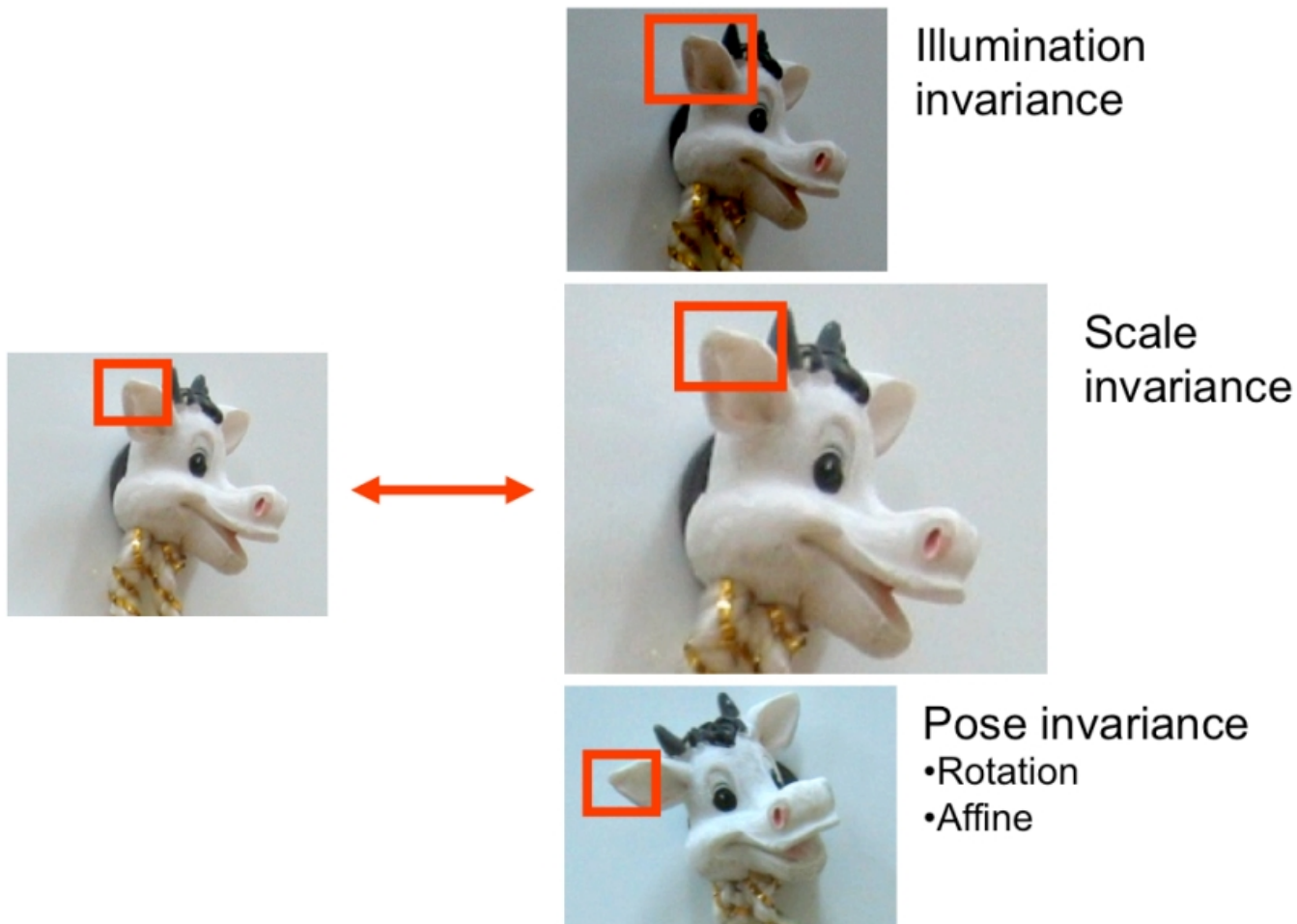
Keypoints or Features

- What keypoints I have to select?



- **Repeatability:** to be able to find the same features on even very different images
- **Saliency:** the feature contains information
- **Locality:** relatively small area of the image, namely more robust wrt occlusions and clutter

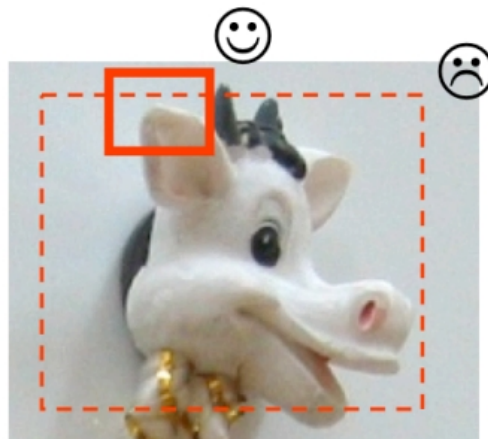
Repeatability



- Saliency

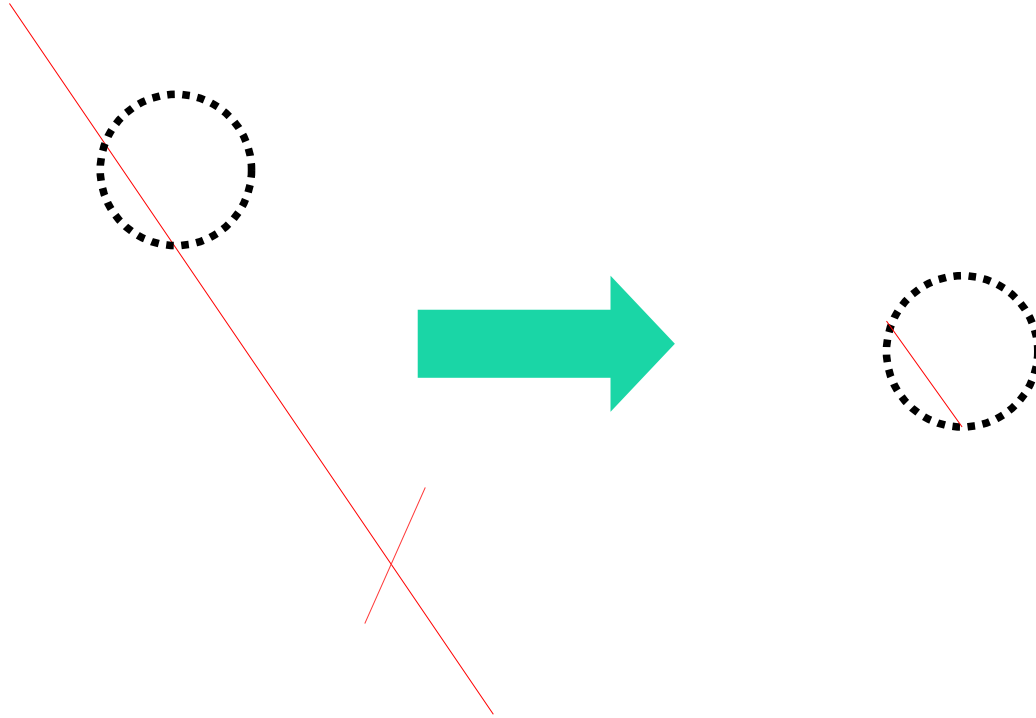


- Locality

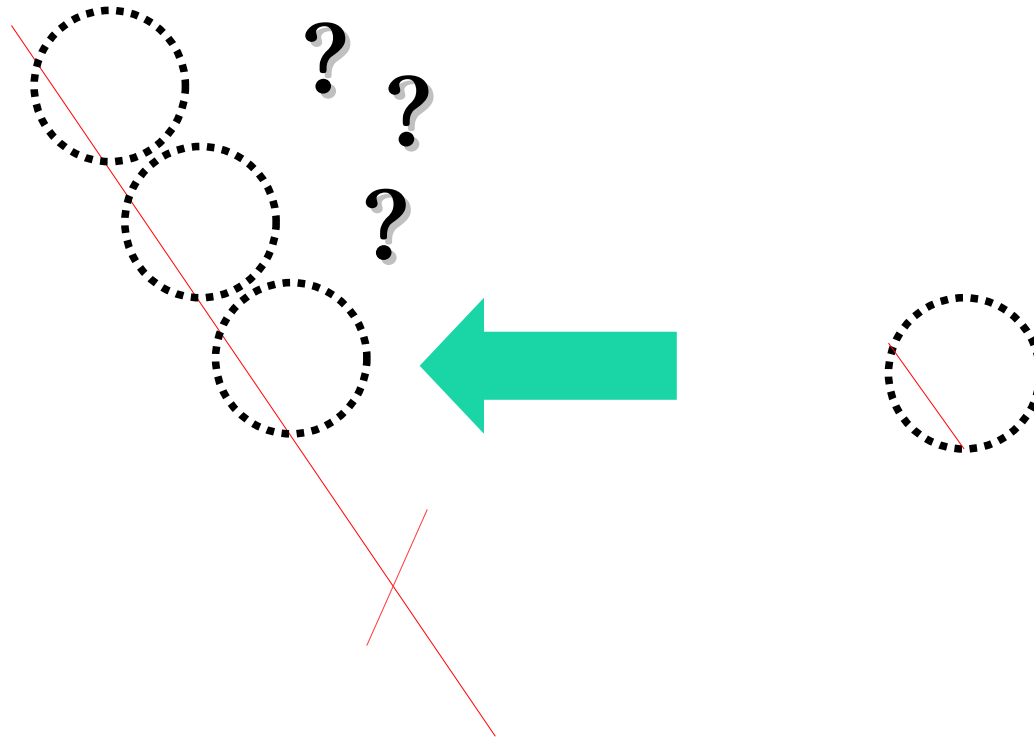


- Textureless patches are nearly impossible to match
- Patches with large contrast changes (gradients) are easier to localize
 - We want edges!
- Are straight edges a good option?
 - Aperture Problem

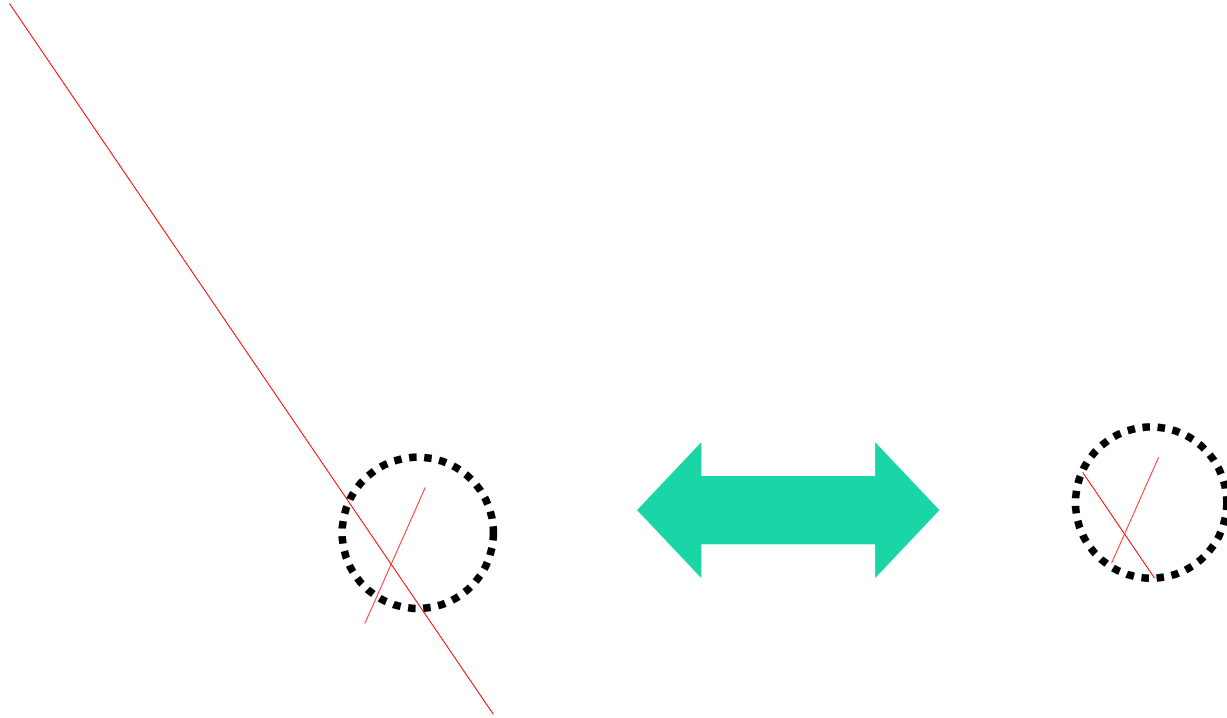
Aperture problem



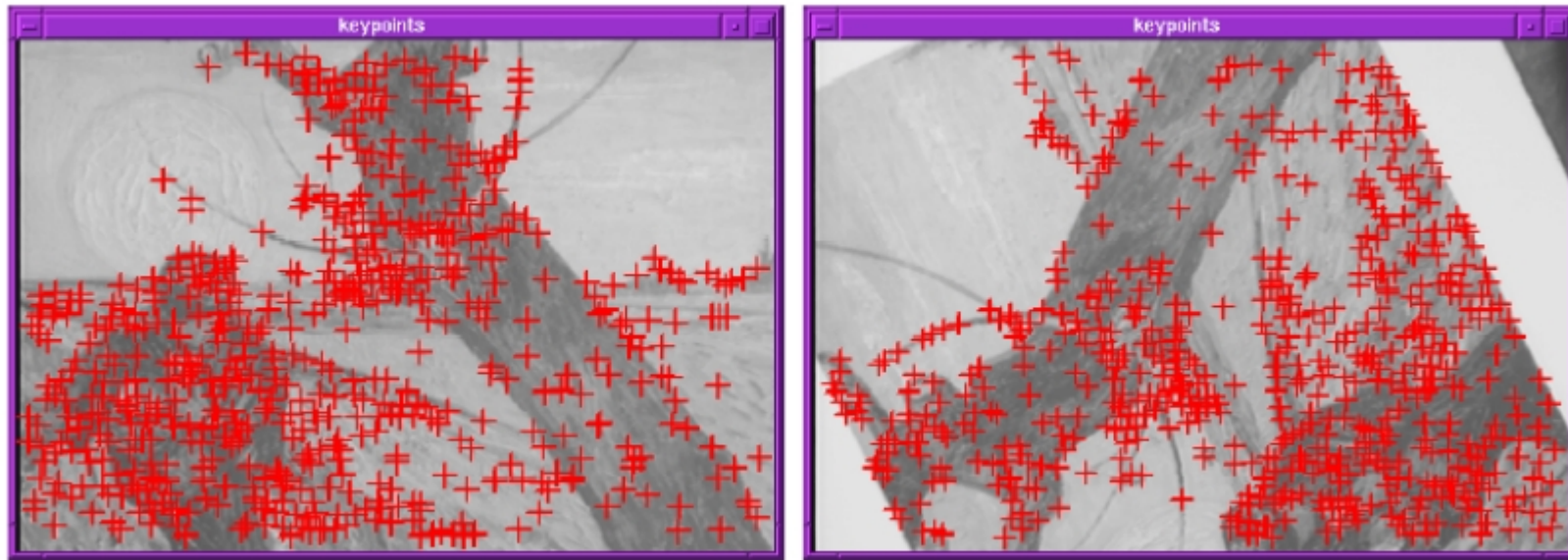
Aperture problem



Aperture problem

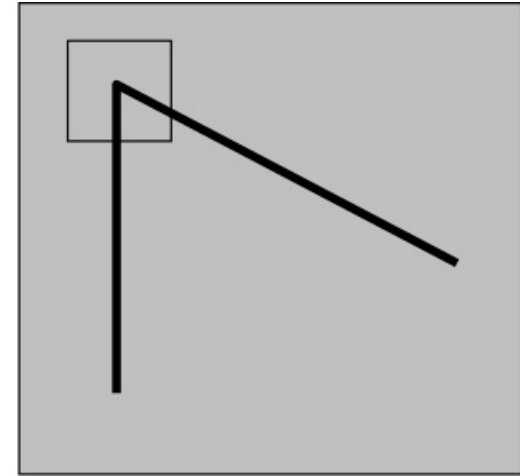
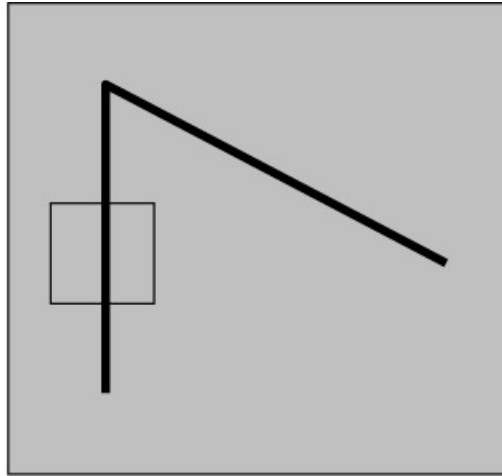
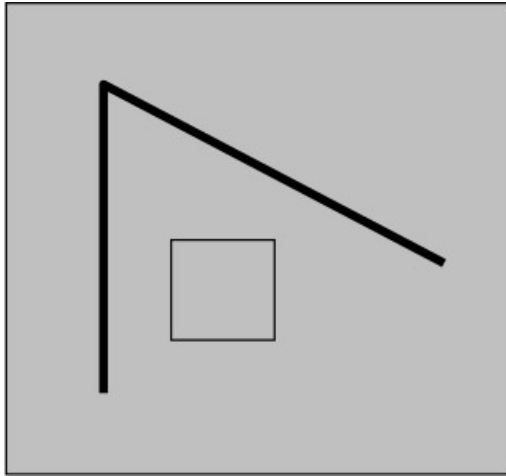


- In “real” images a corner is easier to match
 - 2 “strong” gradients

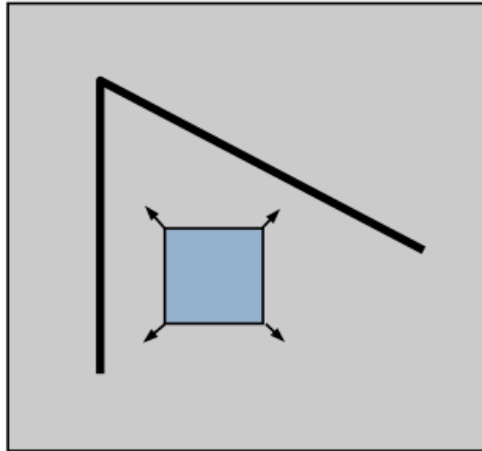


Repeatability – Saliency – Locality

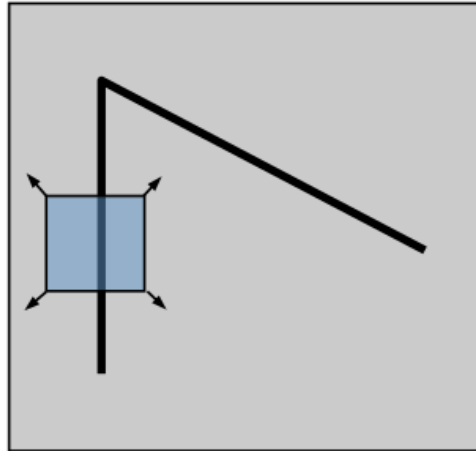
- What defines whether our patch is a good or bad candidate?



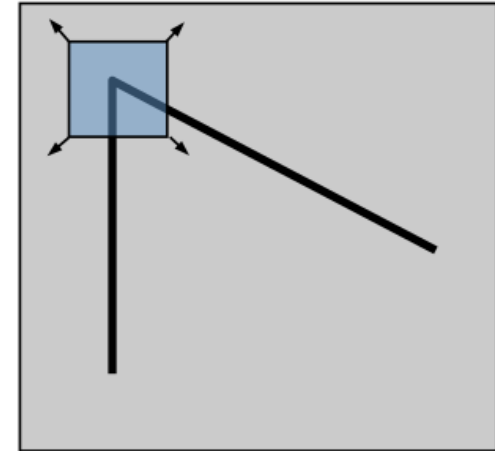
- Let's move a small window (W) on the image
- How the content is changing?



“flat” region:
no change in all
directions



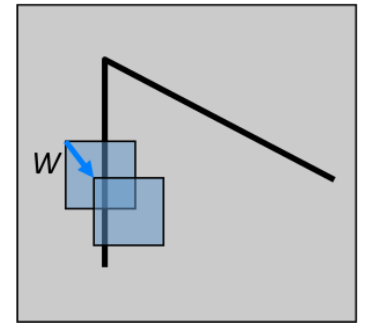
“edge”:
no change along the
edge direction



“corner”:
significant change in
all directions

- Let's move window W by (u,v)
 - u and v are small...
- A good estimation can be the Sum of Squared Differences (SSD)
- The SSD error $E(u, v)$ can be defined as

$$E(u, v) = \sum_{(x, y) \in W} [I(x+u, y+v) - I(x, y)]^2$$



- Using a Taylor Expansion we can compute $I(x+u, y+v)$

$$I_x = \frac{\partial I(x, y)}{\partial x} \quad I_y = \frac{\partial I(x, y)}{\partial y}$$

$$I(x+u, y+v) = I(x, y) + \frac{I_x u}{1!} + \frac{I_y v}{1!} + \text{higher order terms}$$

$$\approx I(x, y) + I_x u + I_y v$$

$$\approx I(x, y) + \begin{bmatrix} I_x & I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

- And then replace it in $E(u,v)$ $I_x = \frac{\partial I(x,y)}{\partial x}$ $I_y = \frac{\partial I(x,y)}{\partial y}$

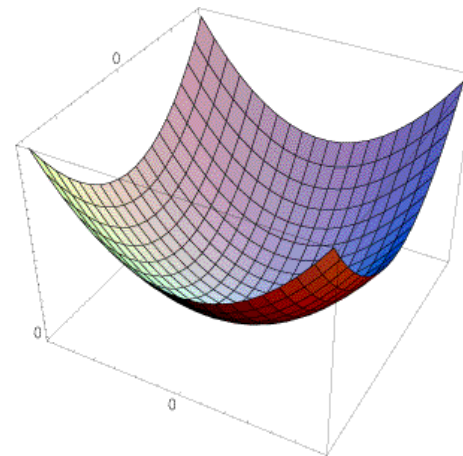
$$\begin{aligned} E(u,v) &= \sum_{(x,y) \in W} \left[I(x+u, y+v) - I(x,y) \right]^2 \\ &\approx \sum_{(x,y) \in W} \left[I(x,y) + I_x u + I_y v - I(x,y) \right]^2 \\ &\approx \sum_{(x,y) \in W} \left[I_x u + I_y v \right]^2 \end{aligned}$$

- Result

$$I_x = \frac{\partial I(x, y)}{\partial x} \quad I_y = \frac{\partial I(x, y)}{\partial y}$$

$$E(u, v) \approx \sum_{(x, y) \in W} [I_x u + I_y v]^2$$

$$\approx Au^2 + 2Buv + Cv^2$$



$$A = \sum_{(x, y) \in W} I_x^2 \quad B = \sum_{(x, y) \in W} I_x I_y \quad C = \sum_{(x, y) \in W} I_y^2$$

- Using a Taylor Expansion $I_x = \frac{\partial I(x, y)}{\partial x}$ $I_y = \frac{\partial I(x, y)}{\partial y}$

$$E(u, v) \approx \sum_{(x, y) \in W} [I_x u + I_y v]^2$$

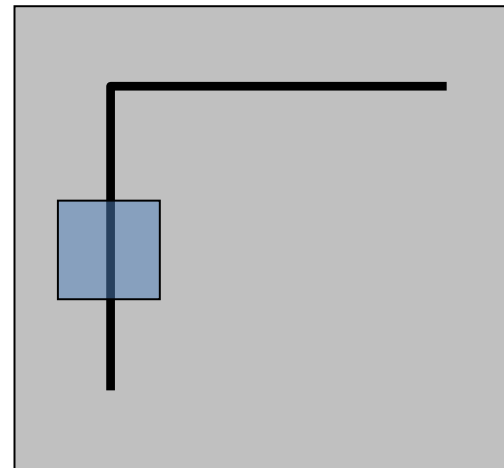
$$\approx Au^2 + 2Buv + Cv^2$$

$$\approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix}$$

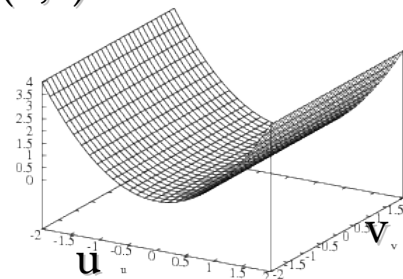
- Vertical Edge $\rightarrow I_y=0$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$\approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$



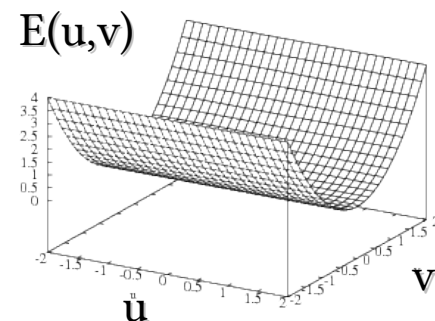
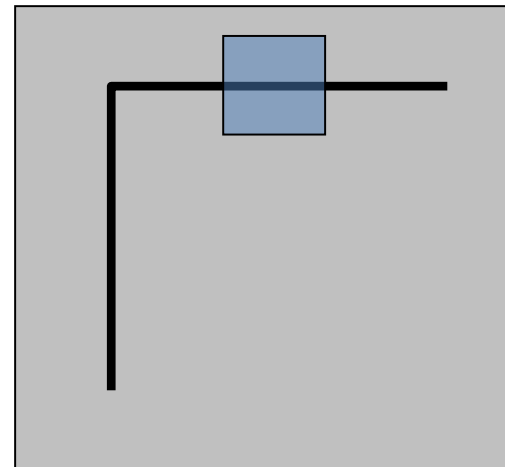
$E(u, v)$



- Horizontal Edge $\rightarrow I_x=0$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

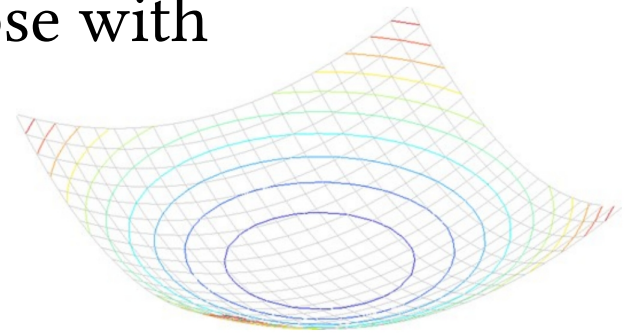
$$\approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$



- General Case

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix}$$

- H can be considered as describing an ellipse with
 - Axis length $\propto$ H eigenvalues
 - Axis orientation $\rightarrow$ H eigenvectors



Eigenvector/Eigenvalues review

- Given a matrix A eigenvectors x and eigenvalues λ satisfy:

$$Ax = \lambda x$$

- Eigenvalues can be found solving

$$\det(A - \lambda I) = 0$$

- For a 2×2 matrix like H we can find $\lambda \rightarrow \det \begin{bmatrix} h_{11} - \lambda & h_{12} \\ h_{12} & h_{22} - \lambda \end{bmatrix}$
 - This leads to a **quadratic** equation
 - i.e. λ_1 and λ_2
- Once found λ s, x can be computed as $\rightarrow \begin{bmatrix} h_{11} - \lambda & h_{12} \\ h_{12} & h_{22} - \lambda \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0$

Eigenvector/Eigenvalues review



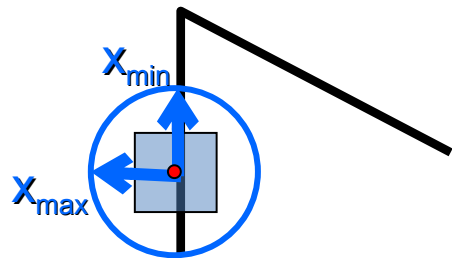
$$\lambda_{1,2} = \frac{1}{2} \left[h_{11} + h_{22} \pm \sqrt{4 h_{12} h_{21} + (h_{11} - h_{22})^2} \right]$$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix}$$

$$Hx_{\max} = \lambda_{\max} x_{\max}$$

$$Hx_{\min} = \lambda_{\min} x_{\min}$$

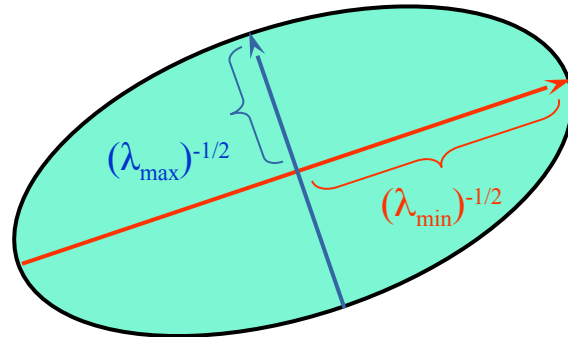
- $x_{\max}$ $\rightarrow$ direction of largest increase for $E(u, v)$
- $x_{\min}$ $\rightarrow$ direction of smallest increase for $E(u, v)$
- $\lambda_{\max}$ $\rightarrow$ amount of increase along $x_{\max}$
- $\lambda_{\min}$ $\rightarrow$ amount of increase along $x_{\min}$



- H can be considered as describing an ellipse with
 - Axis length $\propto$ H eigenvalues
 - Axis orientation $\rightarrow$ H eigenvectors

direction of the fastest
change

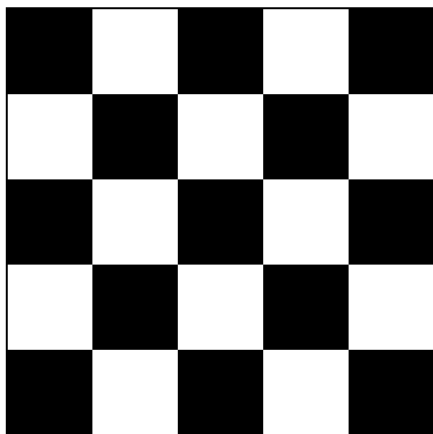
$\lambda_{\max}$, $\lambda_{\min}$: eigenvalues of H



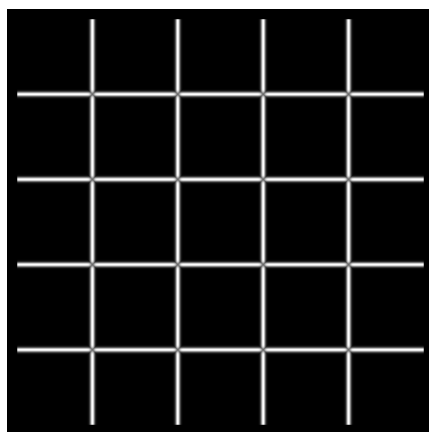
direction of the slowest change

$$H = R \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R^{-1}$$

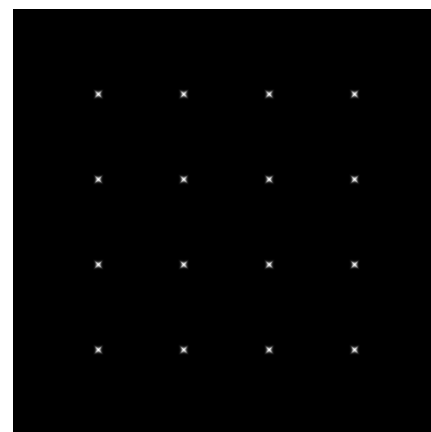
- How $\lambda_{\min}$ and $\lambda_{\max}$ behave on corners?
- $\lambda_{\max}$ is “big” when we have at least one “strong” gradient
- $\lambda_{\min}$ encodes the strenght of the ortogonal gradient



I



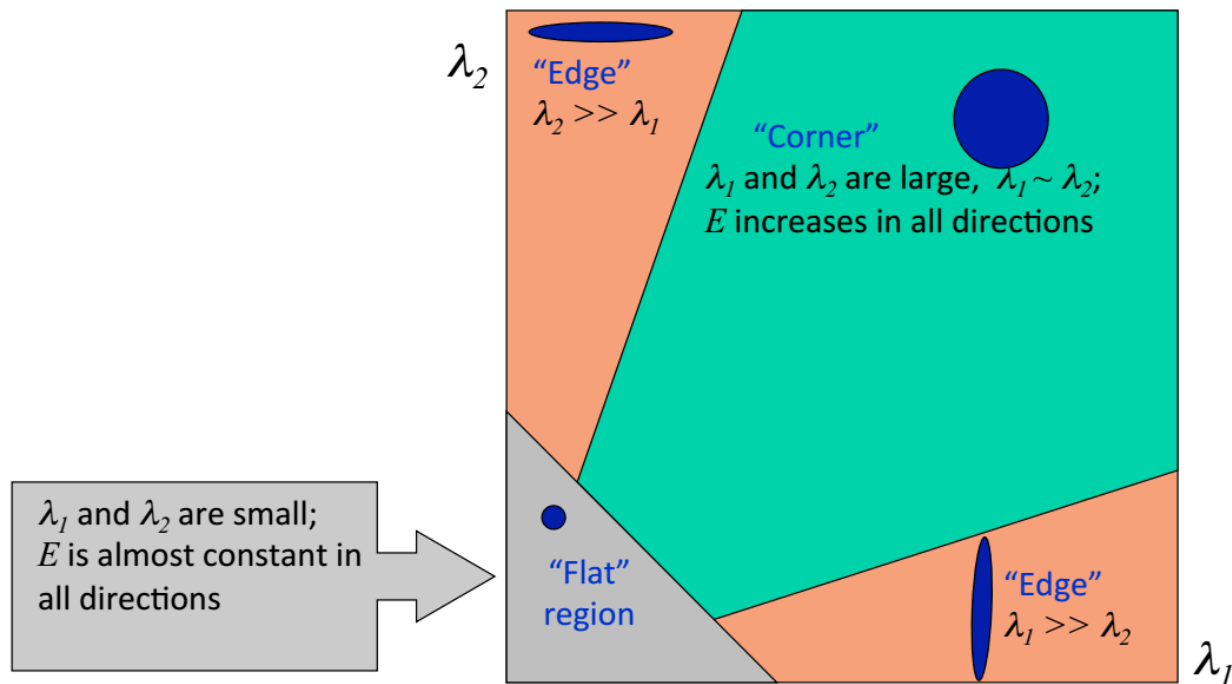
$\lambda_{\max}$



$\lambda_{\min}$

- In a corner both λ_1 and λ_2 are “large”

Slide credit: Kristen Grauman



- Summary

- Compute horizontal and vertical gradients

- For each pixel

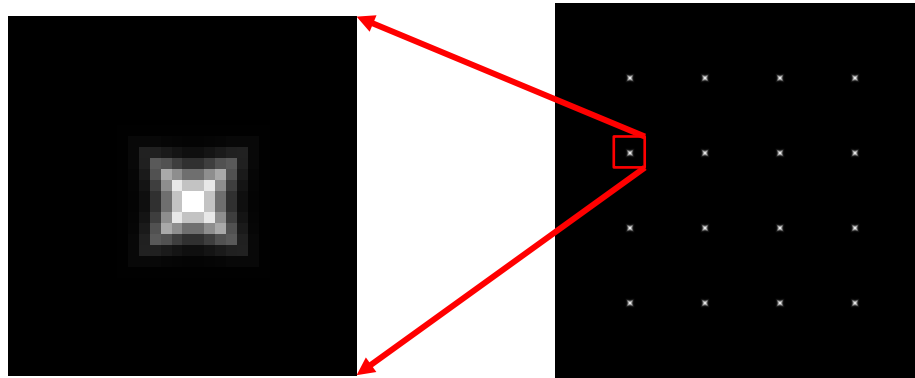
- Create H matrix from gradient

- Compute H eigenvalues

- Discard pixels with $\lambda_{\min} \ll \lambda_{\max} < \text{threshold}$

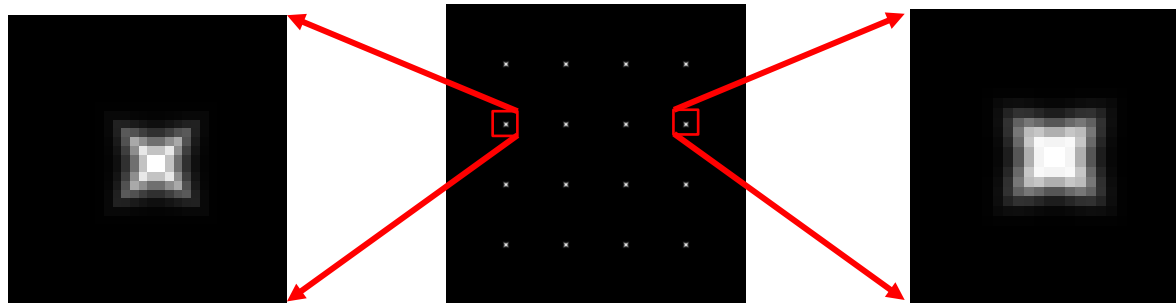
- Apply a NMS approach to find pixels where $\lambda_{\min}$ is a local maximum

$$H = \begin{bmatrix} \sum_{(x,y) \in W} I_x^2 & \sum_{(x,y) \in W} I_x I_y \\ \sum_{(x,y) \in W} I_x I_y & \sum_{(x,y) \in W} I_y^2 \end{bmatrix}$$



- “Harris” is a little variant of previous detector
 - No need to compute λ s $\rightarrow$ no square roots
 - Weight function $\rightarrow$ based on the distance from the center pixel

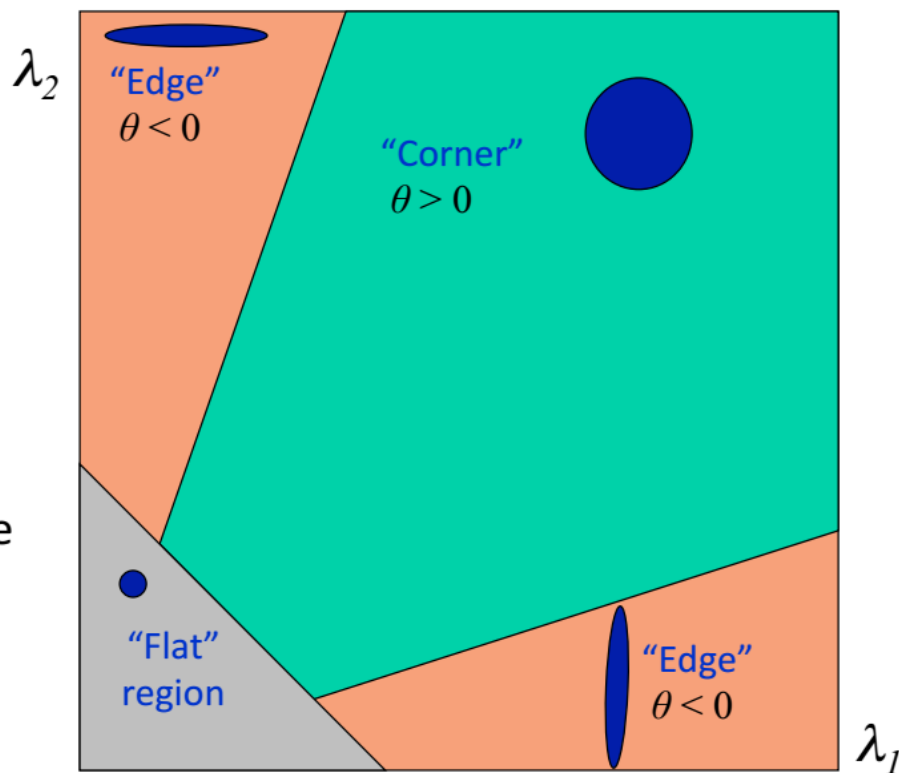
$$\theta = \det(H) - \alpha \text{trace}(H)^2$$



$$\theta = \det(M) - \alpha \text{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

Slide credit: Kristen Grauman

- Fast approximation
 - Avoid computing the eigenvalues
 - α : constant (0.04 to 0.06)



- Shi et al. (1994) $\lambda_{min}^{\frac{1}{2}} \rightarrow$
- Harris et al. (1998) $\rightarrow \theta = \det(H) - \alpha \text{trace}(H)^2 = \lambda_0 \lambda_1 - \alpha (\lambda_0 + \lambda_1)^2$
- Triggs et al. (2004) $\rightarrow \lambda_{min} - \alpha \lambda_{max}$
- Brown et al. (2005) $\rightarrow \lambda_{min} \approx \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2} = \frac{\det(H)}{\text{trace}(H)}$

- Harris also introduced a window function
 - The idea is to weight distance

$$E(u, v) = \sum_{(x, y) \in W} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Window
Function

Shifted
Intensity

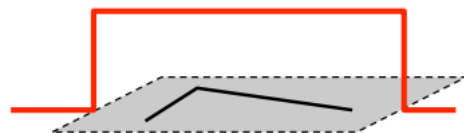
Intensity

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- Option 1: uniform window
 - Sum over square window

$$M = \sum_{x,y} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- Problem: not rotation invariant

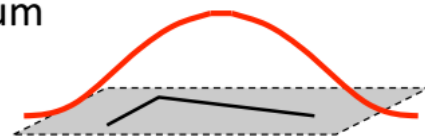


1 in window, 0 outside

- Option 2: Smooth with Gaussian
 - Gaussian already performs weighted sum

$$M = g(\sigma) * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- Result is rotation invariant



Gaussian

- Summary

- Compute horizontal and vertical gradients

- Apply window function

- For each pixel

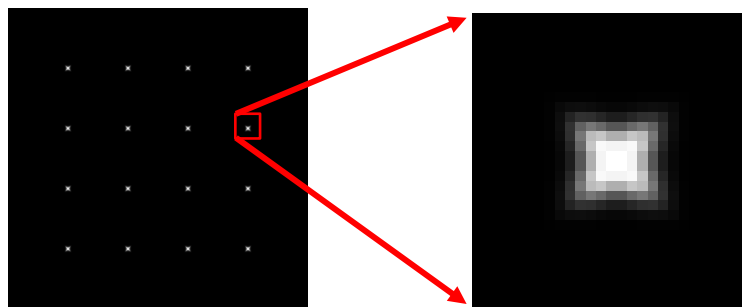
- Create H matrix from gradients

- Compute θ

- Discard pixels with $\theta < 0$ or anyway below a given threshold

- Apply a NMS approach to find pixels where $\lambda_{\min}$ is a local maximum

$$H(\sigma_I, \sigma_D) = g(\sigma_I) \begin{bmatrix} \sum_{(x,y) \in W} I_x^2(\sigma_D) & \sum_{(x,y) \in W} I_x I_y(\sigma_D) \\ \sum_{(x,y) \in W} I_x I_y(\sigma_D) & \sum_{(x,y) \in W} I_y^2(\sigma_D) \end{bmatrix}$$

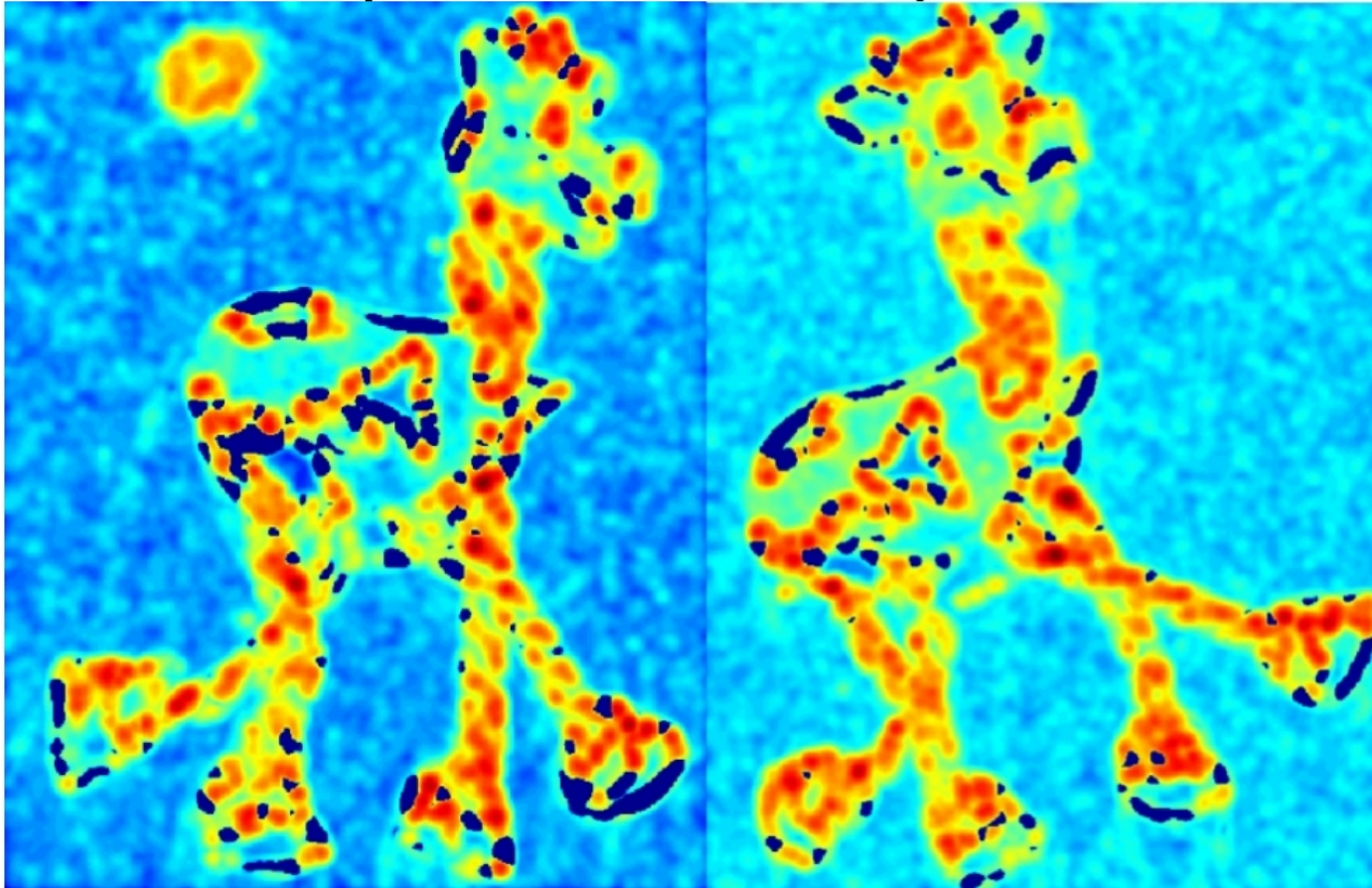


Harris Corner Detector steps



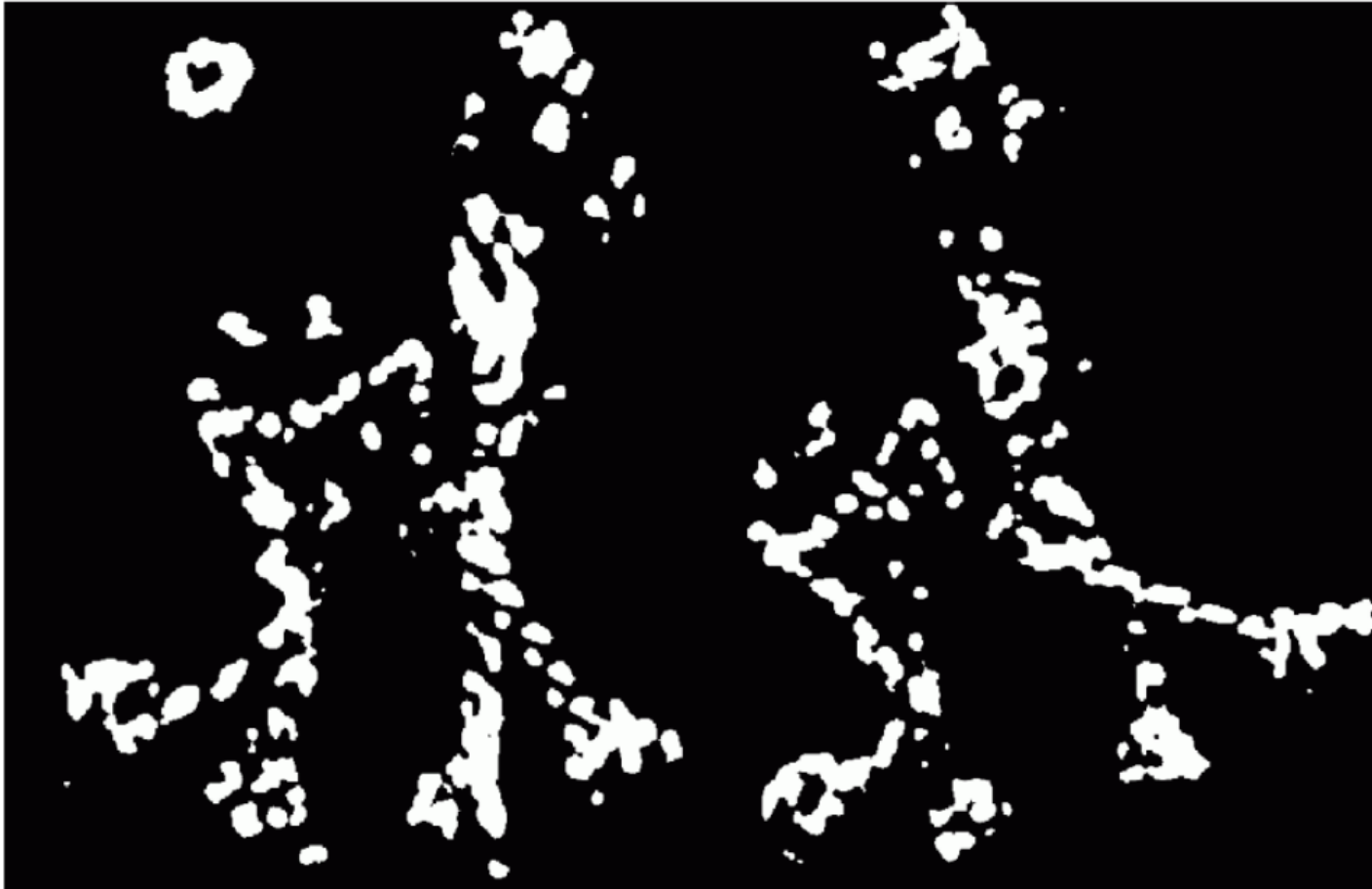
Slide adapted from Darya Frolova, Denis Simakov

Harris Corner Detector steps



Slide adapted from Darya Frolova, Denis Simakov

Harris Corner Detector steps



Harris Corner Detector steps



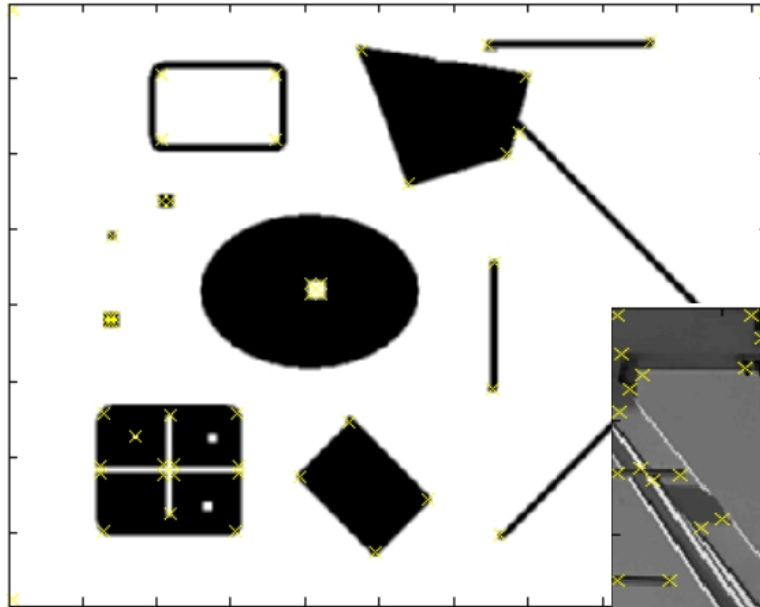
Slide adapted from Darya Frolova, Denis Simakov

Harris Corner Detector steps

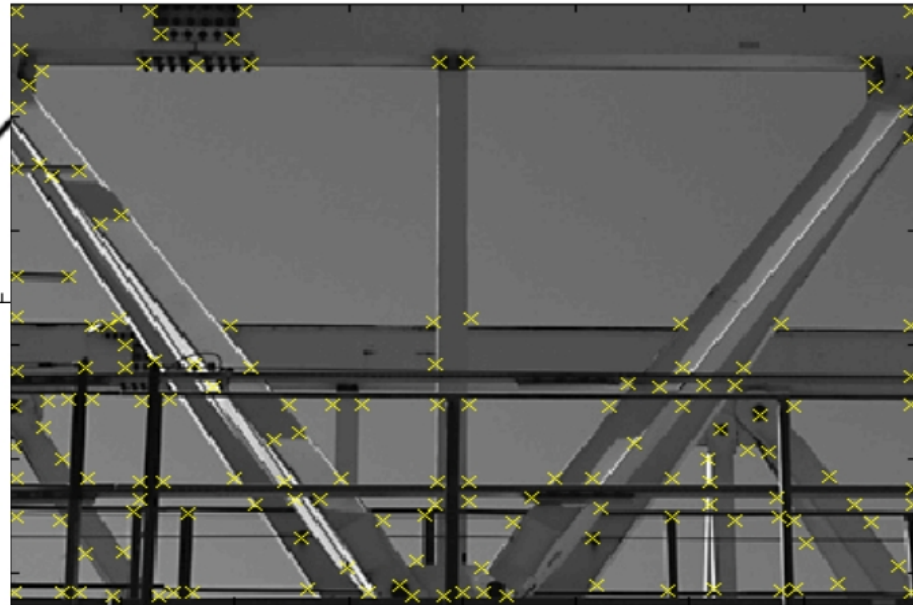


Slide adapted from Darva Frolova, Denis Simakov

Examples



Effect: A very precise
corner detector.



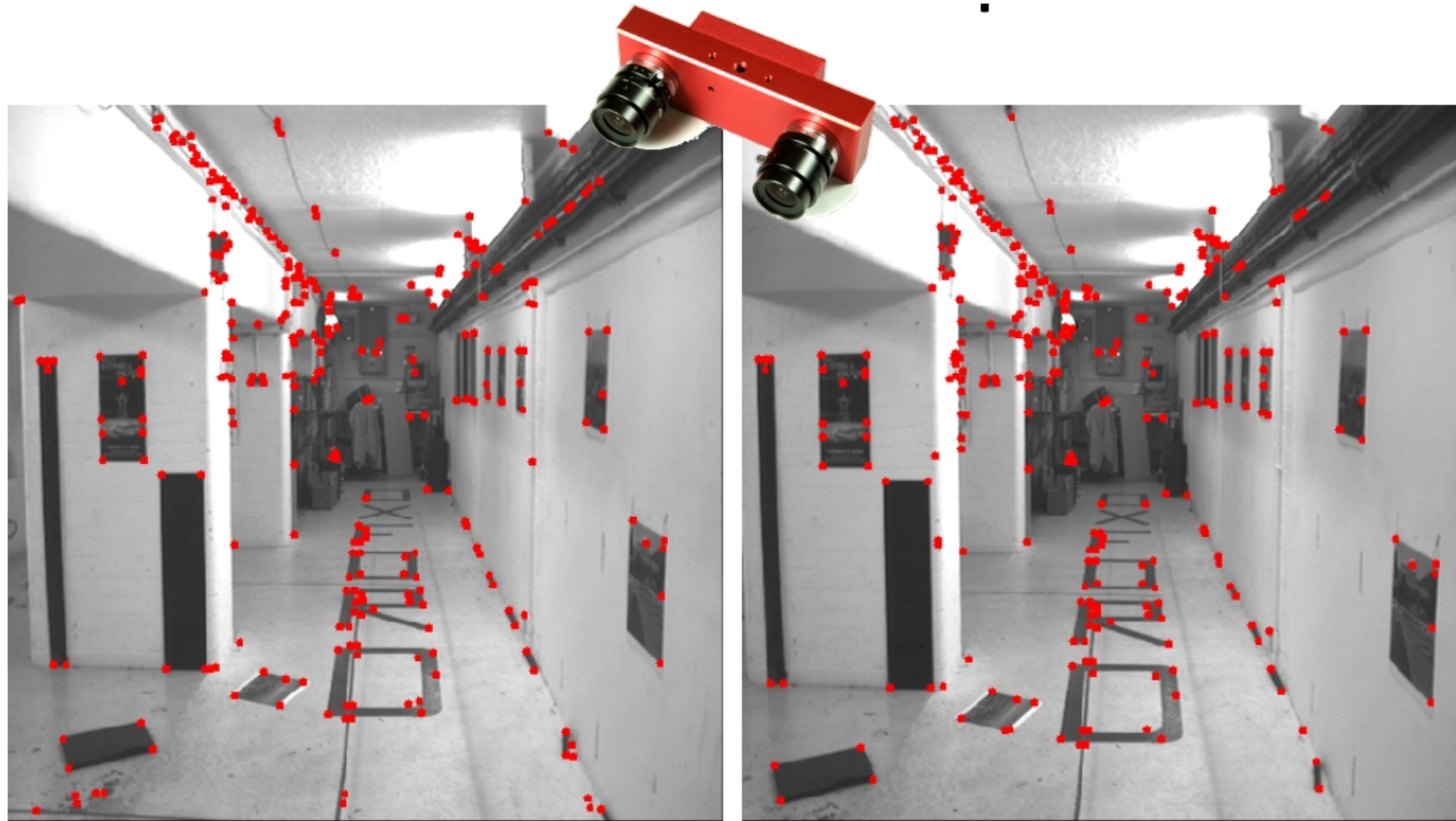
Slide credit: Krystian Mikolajczyk

Examples



Slide credit: Krystian Mikolajczyk

Sparse Stereo



- How much keypoints extraction is robust to image transformations?

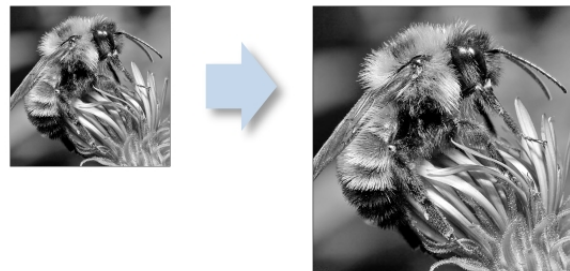
- Rotation (traslation)



- Intensity change

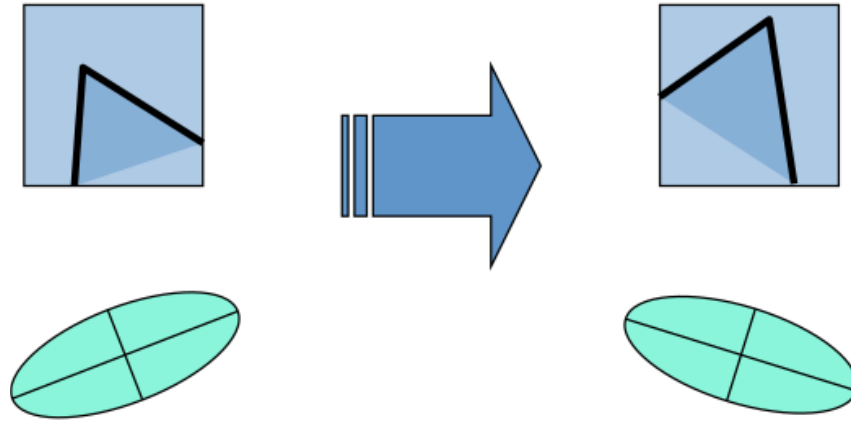


- Scale



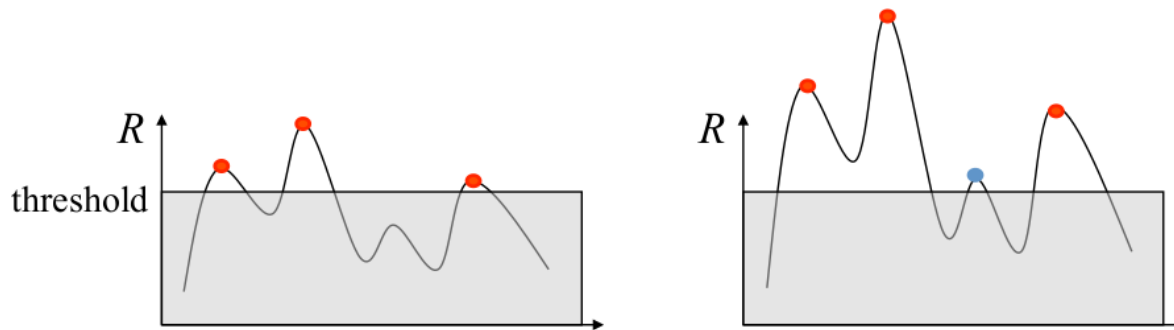
- Ellipse rotates but eigenvalues are the same

$$H = R \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R^{-1}$$



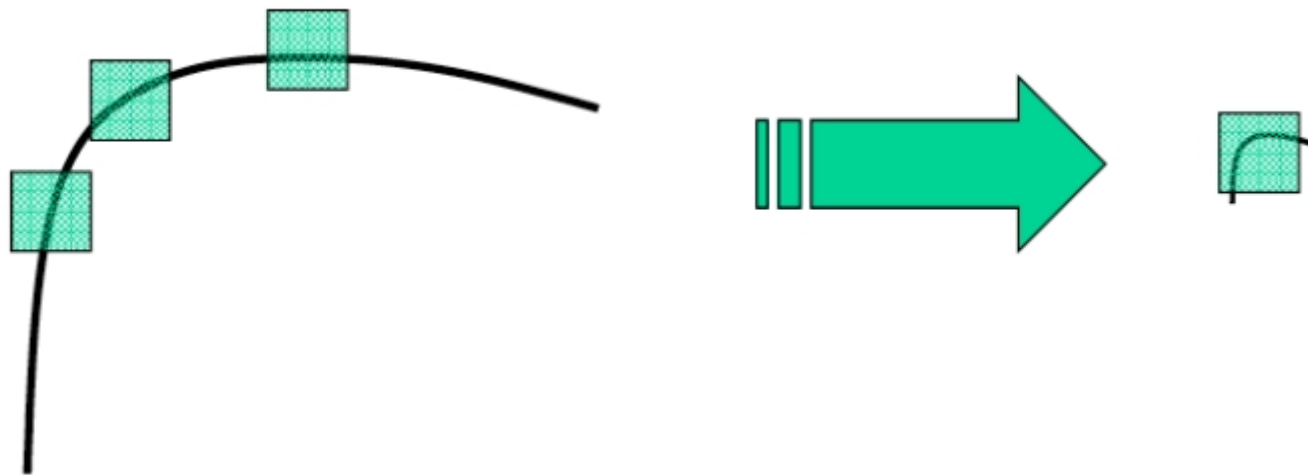
Invariant to Rotation

- Intensity change can be modeled as $g(r, c) = a \cdot f(r, c) + b$
- Derivative operations make result invariant wrt b
- Not so true about a ...



Partially Invariant to Intensity Changes

- Right $\rightarrow$ corner
- Left $\rightarrow$ corner

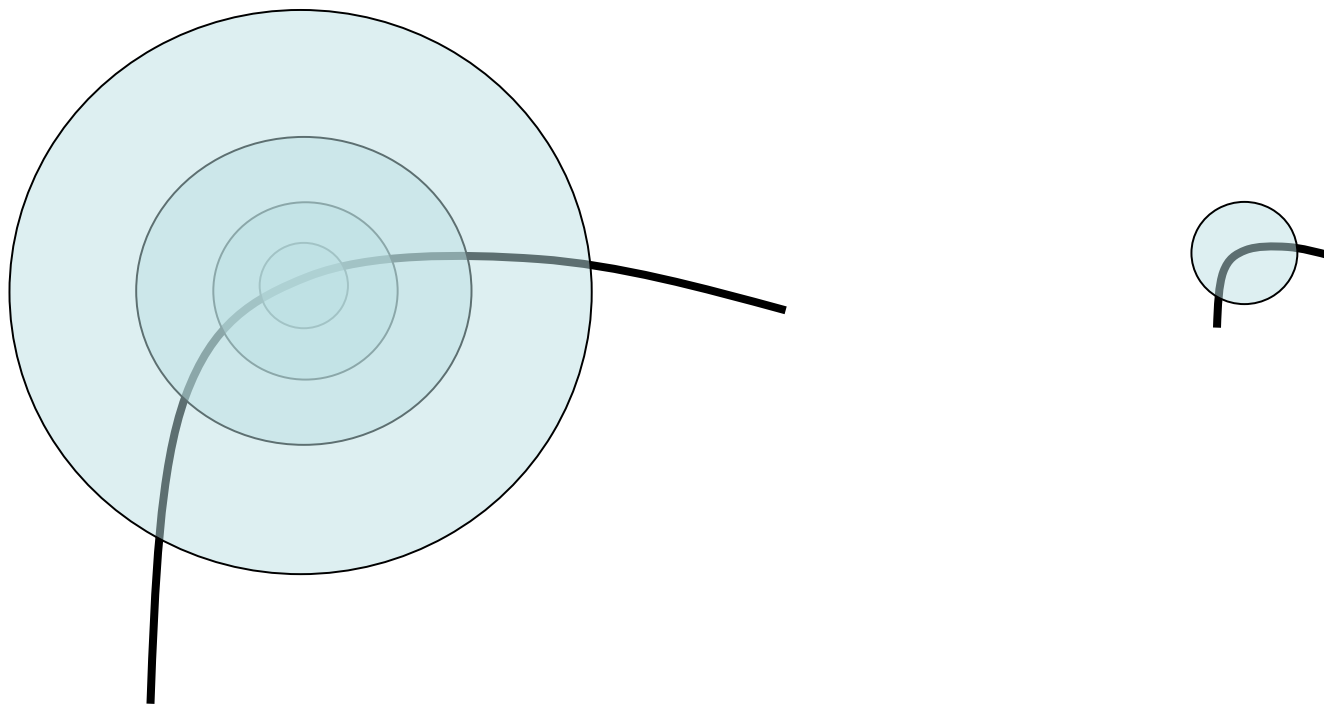


Not Invariant to Scale Changes

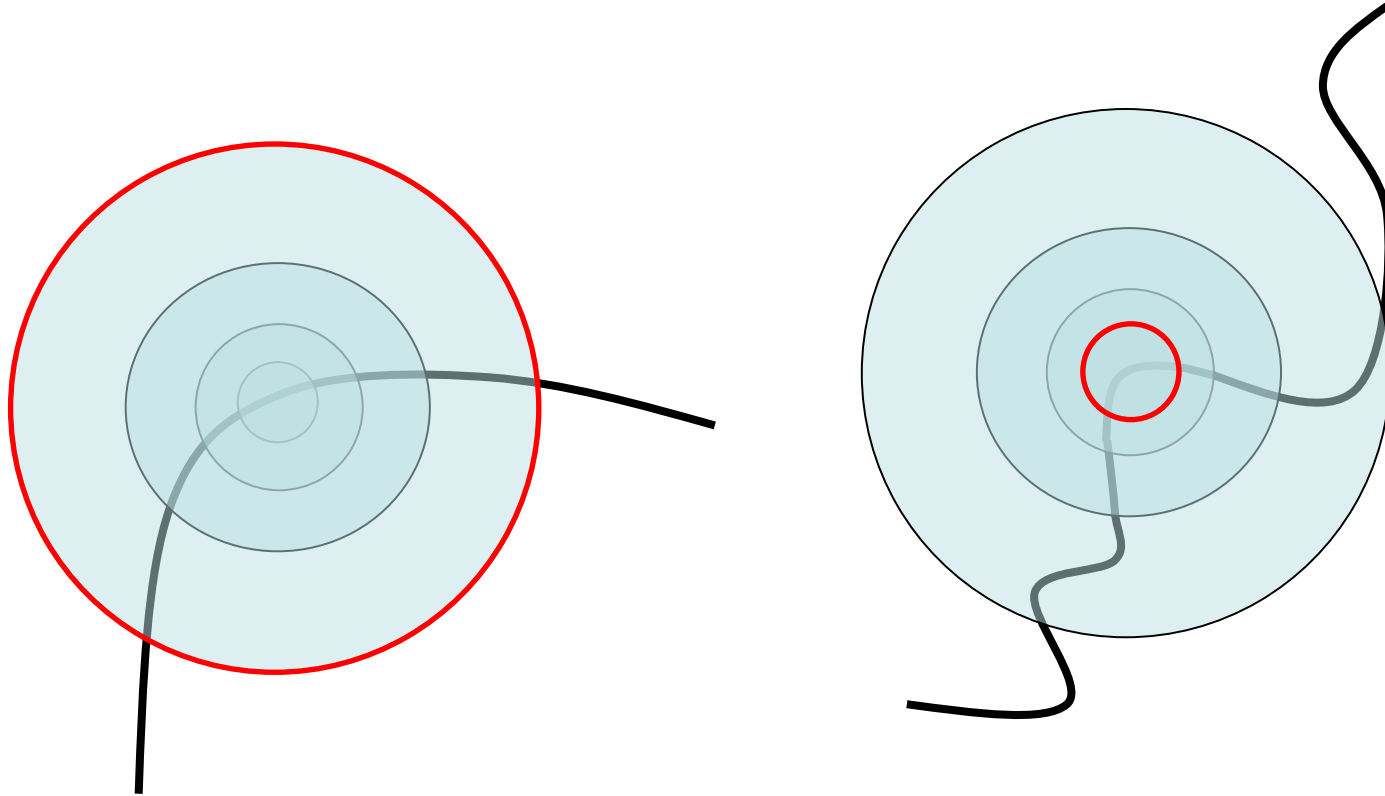
- The keypoints are highly dependent on window size



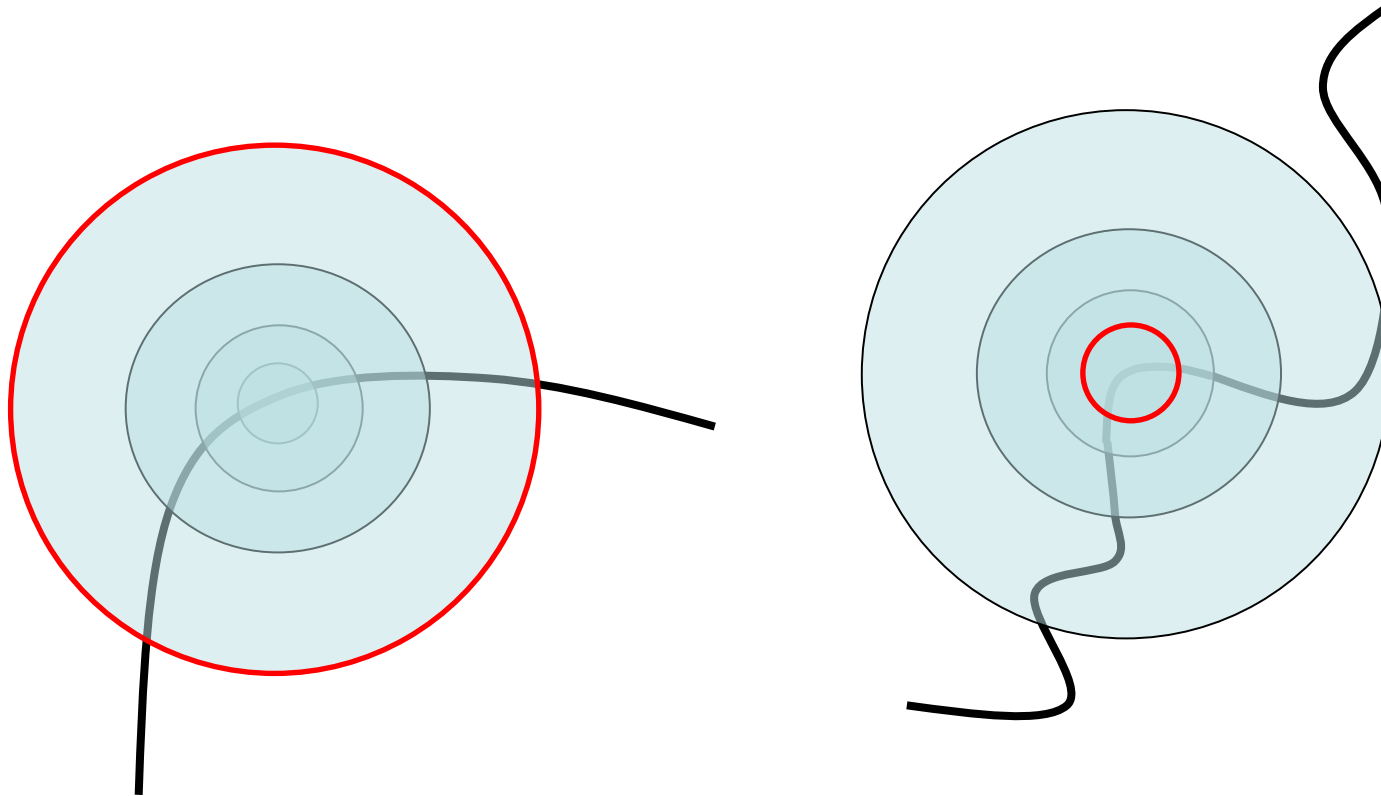
- Consider circles of different size around a point
- We can find a size that look similar in both images



- How do we select appropriate circles size in both images?

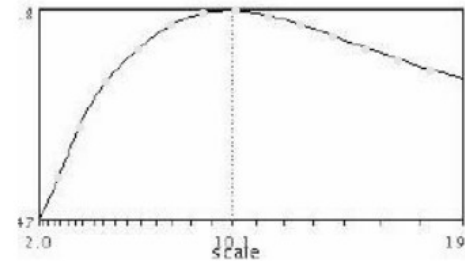
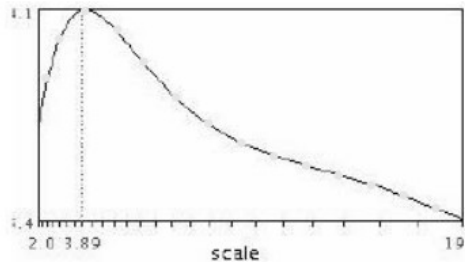


- We can select the window size that **maximize** Harris operator



Scale invariant detector

- Example



- Instead computing θ for larger and larger windows a pyramidal approach can be used



(sometimes need to create in-between levels, e.g. a $\frac{3}{4}$ -size image)

- **HARRIS-LAPLACIAN**
 - K. Mikolajczyk, C. Schmid, Indexing based on scale invariant interest points, ICCV 2001
- **SIFT**: Scale Invariant Feature Transform
 - D. Lowe. “Distinctive Image Features from Scale-Invariant Keypoints“, IJCV 2004
- **FAST**: Features from Accelerated Segment Test
 - Edward Rosten and Tom Drummond, “Machine learning for high speed corner detection” in 9th European Conference on Computer Vision, vol. 1, 2006
- **SURF**: Speeded-Up Robust Features
 - Herbert Bay, Andreas Ess, Tinne Tuytelaars, Luc Van Gool, "SURF: Speeded Up Robust Features", Computer Vision and Image Understanding (CVIU), 2008
- **MSER**: Maximally Stable Extremal Regions
 - J. Matas, O. Chum, M. Urban, and T. Pajdla. "Robust wide baseline stereo from maximally stable extremal regions." Proc. of British Machine Vision Conference, 2002.
- **BRISK**: Binary Robust Invariant Scalable Keypoint
 - Stefan Leutenegger ; Margarita Chli ; Roland Y. Siegwart, 2011 International Conference on Computer Vision



UNIVERSITÀ DI PARMA

Features Extraction

Question time!





UNIVERSITÀ DI PARMA

Features Extraction

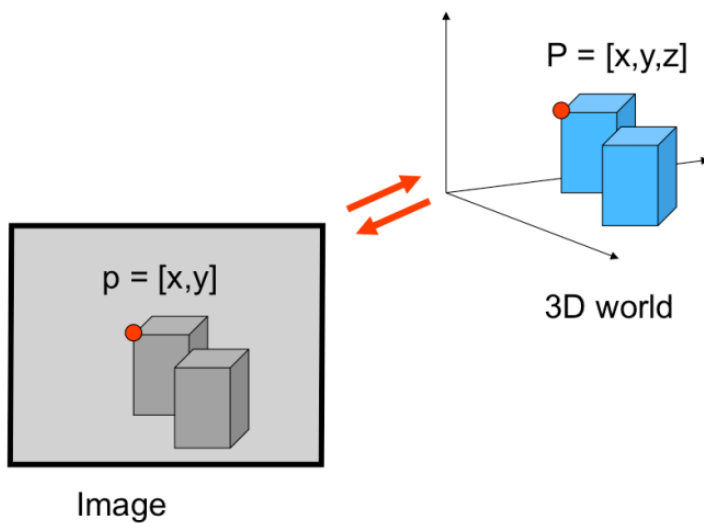


Features Extraction



- What is a feature?
- Keypoints ideal characteristics
- Harris Corners
- Scale Invariant Detectors

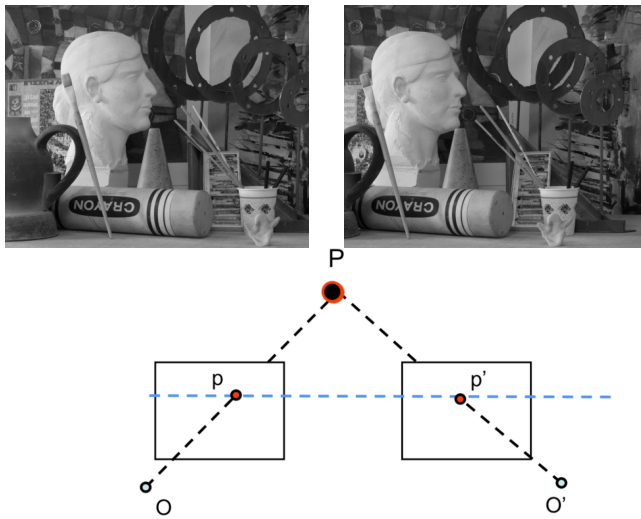
- [FP] D. A. Forsyth and J. Ponce. **Computer Vision: A Modern Approach** (2nd Edition). Prentice Hall, 2011.
- **CS231A · Computer Vision: from 3D reconstruction to recognition**, Prof. Silvio Savarese – Stanford University
- **CS131 · Computer Vision: Foundations and Applications**, Prof. Fei-Fei Li – Stanford University
- **Elementi di Analisi per Visione Artificiale**
 - Paolo Medici <http://www.ce.unipr.it/people/medici>



- We found a geometrical relation between world and image

In precedenza abbiamo visto la relazione che dal mondo 3D ci porta all'immagine 2D.

In questa lezione ci focalizzeremo sull'estrapolare il contenuto visuale dell'immagine



- We found a geometrical relation between world and image

Con lo stereo cerchiamo corrispondenze da una immagine all'altra al fine di calcolare il 3D.

Però tipicamente si sfruttano immagini prese da telecamere relativamente simili e posizionate in maniera di inquadrare la stessa scena



Il match non necessariamente e' solo stereo. Vi sono situazioni in cui cerco corrispondenze in immagini completamente differenti tra di loro.
Le motivazioni possono essere ben differenti dalla semplice ricostruzione 3D



by [Diva Sian](#)



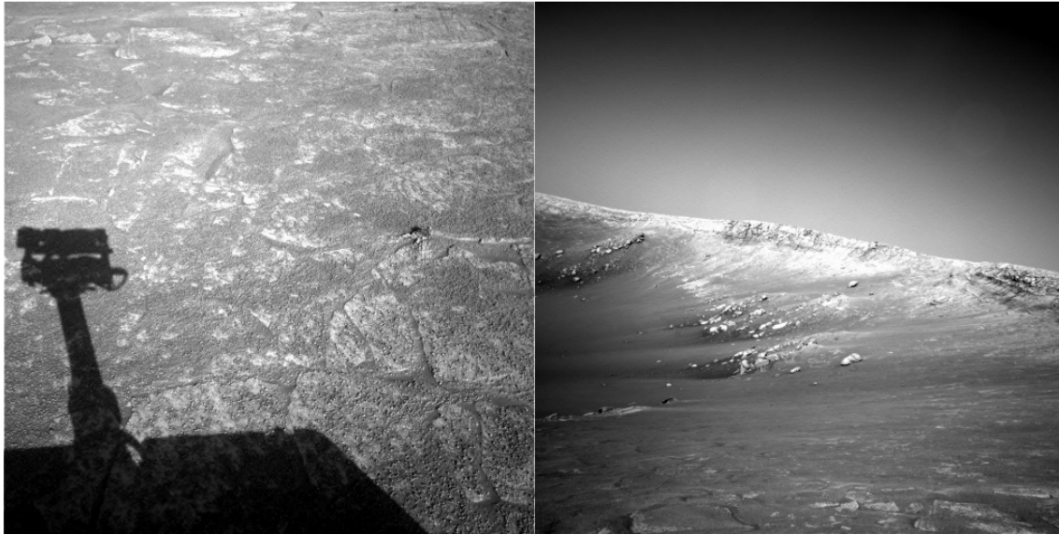
Caso relativamente facile, immagini prese da telecamere differenti ma simili punti di vista



Caso un po' meno semplice

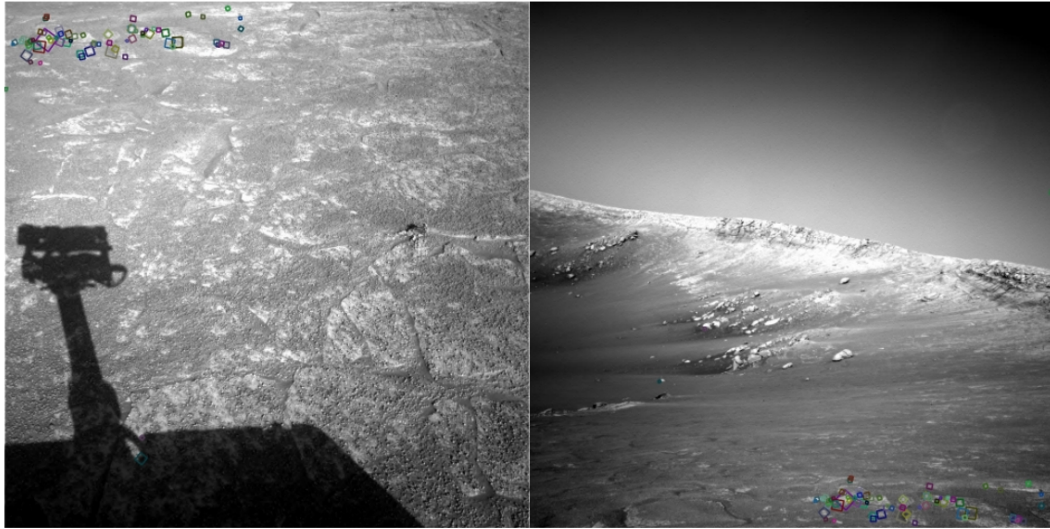


Se io riesco a trovare delle corrispondenze tra immagini anche completamente differenti prese da pose e telecamere differenti. Posso anche ricostruire 3D (ma come detto non è sempre lo scopo principale)



NASA Mars Rover images

Non è un problema semplice. Queste due immagini inquadrano lo stesso scenario. Ora chiaro che quella a sx rappresenta un dettaglio maggiore. Dove si trova quella zona in quella a dx?
Ecco qui non ci riusciamo neanche noi...



NASA Mars Rover images with SIFT feature matches

Non era banale eh?



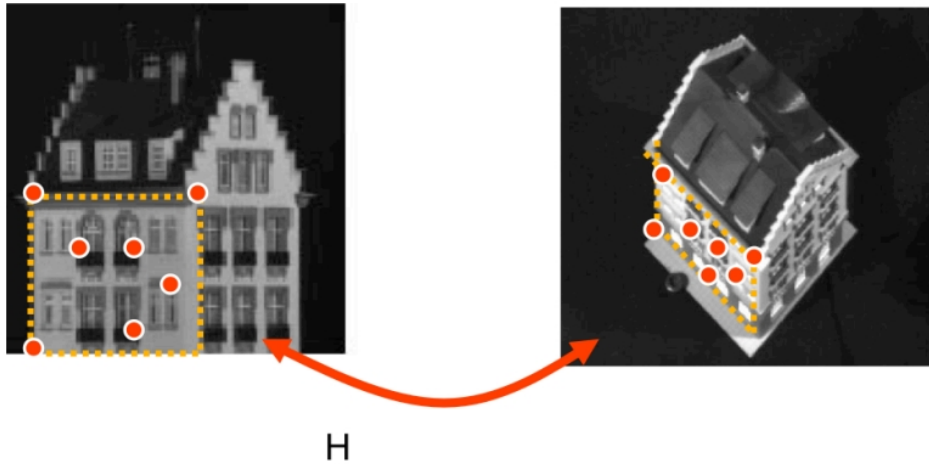
- During this lesson and the following one we are going to learn:
 - What are “features” and their properties
 - How to extract “features” from images
 - How to match them among all images

Il problema è chiaro: capire se in una immagine inquadro qualcosa che c'è in altre. E dove sono queste porzioni di immagini.

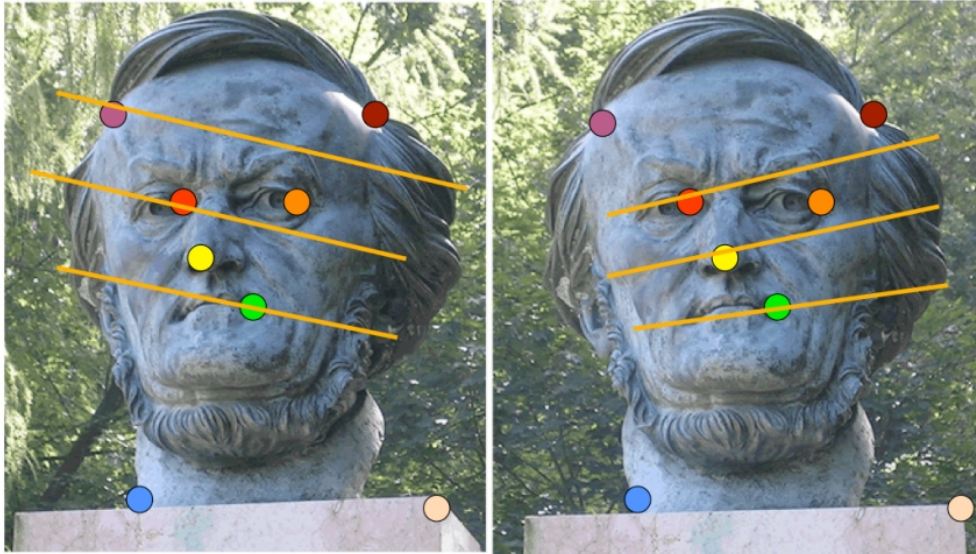
A differenza dello stereo ci limiteremo a parti ben precisi dell'immagine: le feature

- Potential outcomes are:
 - Fit or estimate models that describe an image or a portion of it
 - Match or index images
 - Detect objects or actions from images
 - “Combine” different images

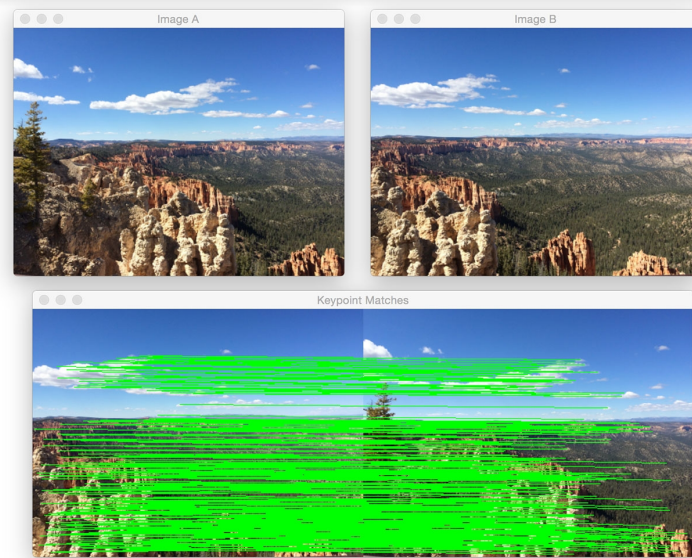
Alcuni esempi di applicazioni



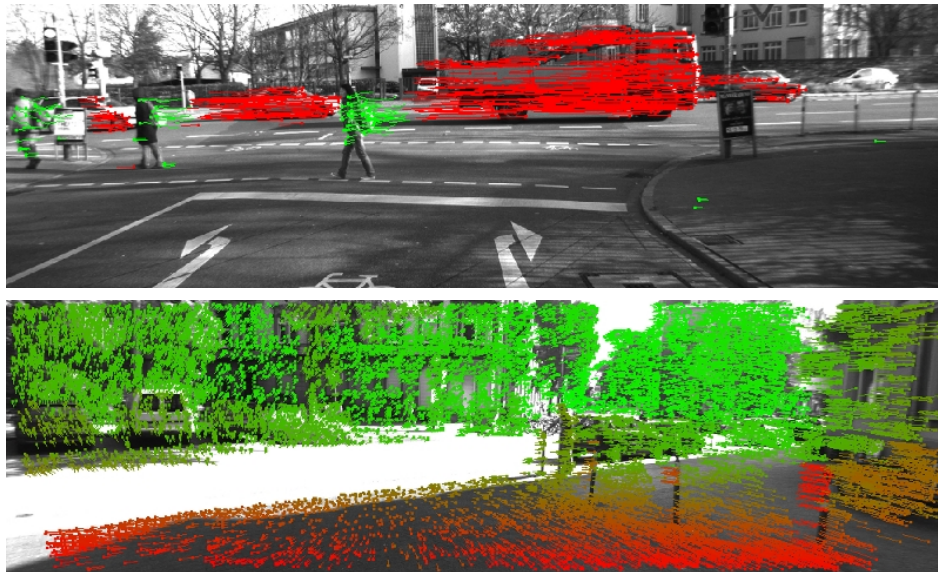
Ad esempio posso usare le corrispondenze tra due immagini per calcolare una omografia



O trovare gli 8 punti per calcolare la matrice fondamentale (o i 5 per quella essenziale)



Stereo sparso?



Se lo applico a sequenze di immagini posso calcolarmi il cosiddetto “flusso ottico”. Qui due esempi con telecamera fissa e con telecamera mobile. Si vede quanto si muovono i punti da una immagine all'altra e in che direzione.



Esempio di flusso ottico



Image stitching ovvero prendo tante immagini della stessa scena e poi le fondo insieme



- Theoretically we could use stereo vision techniques
 - Too much expensive!
 - Moreover, we do not have to obtain a full 3D reconstruction
- Just look at the following case
 - Joining of different images...

Quando le immagini sono troppo differenti tra di loro diventa improponibile la visione stereo per come l'abbiamo vista

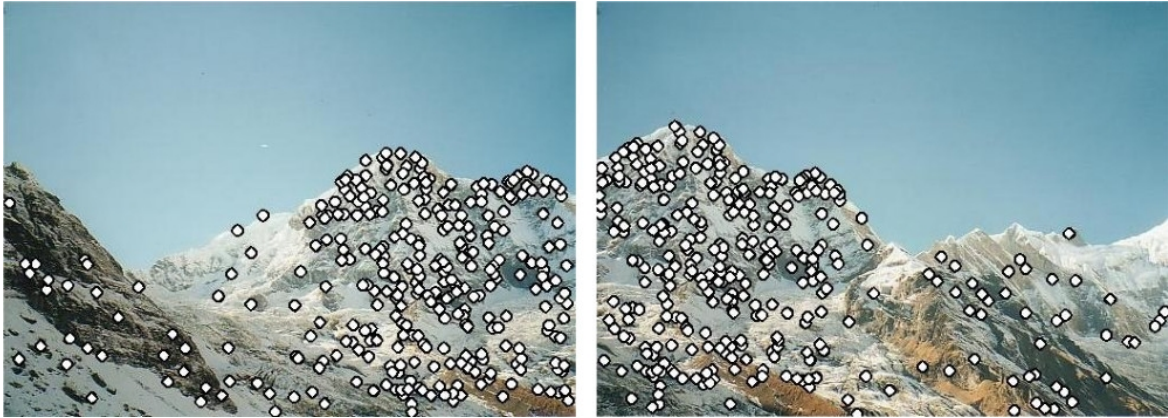


- How to combine following images?



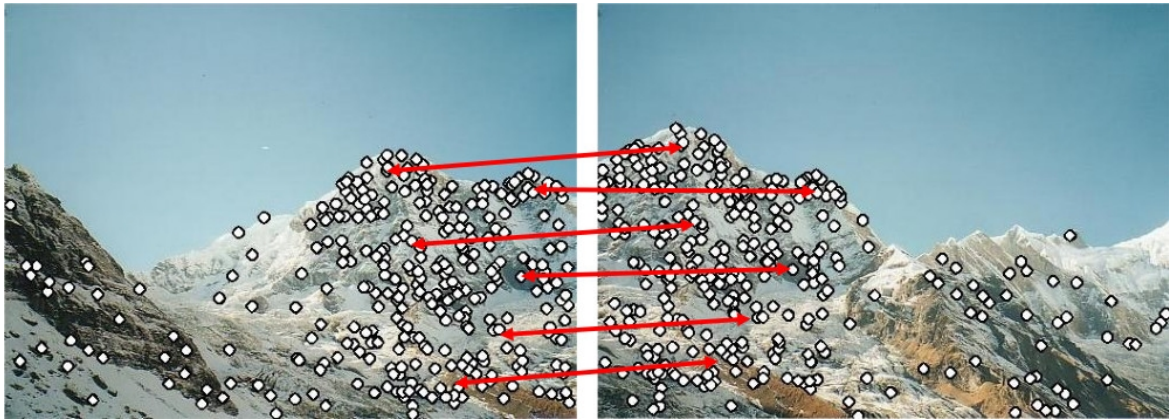
Ma poi non è detto che sia in realtà necessaria, proviamo a capire come combinare le due immagini in questione

- 1. select “specific” points



Devo trovare tutte le corrispondenze possibili dei punti della prima immagine rispetto alla seconda come visto nel match stereo? No. Mi posso limitare ad alcuni punti.

- 2. find potential correspondences



Perché di fatto mi servono poche corrispondenze (ricordiamo che l'omografia è una matrice 3×3)

- 3. compute alignement



Proietto riallineando

- Select specific points → keypoints or features → **features extraction**
- Find potential correspondences → **features matching**
- Application dependent step
 - Matching
 - Indexing
 - Detection
 - Image alignment

Per fare quindi quell'operazione devo selezionare un set di punti.
Vedremo in queste slide che caratteristiche devono avere e come sceglierli

Lo faccio su entrambe le immagini e poi capisco quali siano quelli corrispondenti
Ma questo lo vediamo nel prossimo pacco di slide

- For Stereo Vision
 - Window match → SAD
- In a more general situation
 - Different size
 - Different orientation
- A complete match is impracticable
 - Only keypoints → patches

Non possiamo fare la SAD (o match simile) in quanto dovrò confrontare immagini molto differenti in cui i vari elementi inquadrati possono avere dimensione e orientazione significativamente diverse.

I punti che andremo a selezionare sono i keypoint o feature. Che poi non sono un punto solo ma un area attorno a un punto particolare. Queste aree le indichiamo come patch. Spesso sono rettangolari ma più spesso sono di fatto “circolari”

Attenzione che io userò keypoint, feature e patch con lo stesso significato.

The question we will be asking in this lecture is - how do we represent images? There are many basic ways to do this. We can just characterize them as a collection of pixels with their intensity values. Another, more practical, option is to describe them as a collection of components or features which correspond to “interesting” regions in the image such as corners, edges, and blobs. Each of these regions is characterized by a descriptor which capture the local distribution of certain photometric properties such as intensities, gradients, etc.

- What keypoints I have to select?



Quindi non faccio come con lo stereo che confronto tutti i punti di un'immagine con l'altra ma mi limito a selezionarne solo alcuni.

Spoiler: non li selezioniamo a caso

Ok, ma allora quali selezioniamo e perché?

- **Repeatability:** to be able to find the same features on even very different images
- **Saliency:** the feature contains information
- **Locality:** relatively small area of the image, namely more robust wrt occlusions and clutter

Un buon keypoint deve avere caratteristiche ben precise

Ripetibili

La stessa feature deve essere identificabile su immagini diverse, anche se ha subito trasformazioni geometriche o fotometriche

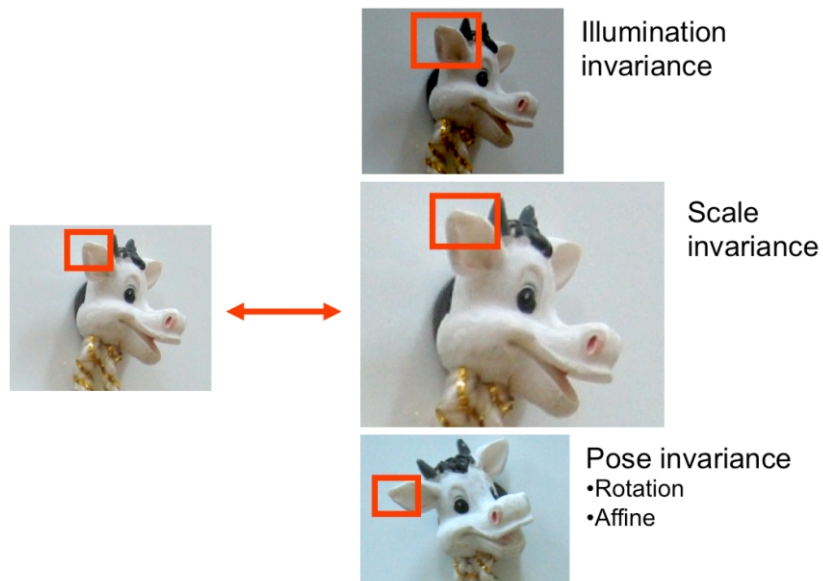
Salienti

Devono essere punti «interessanti», dove si concentra informazione

Locali

Devono occupare regioni dell'immagine relativamente piccole

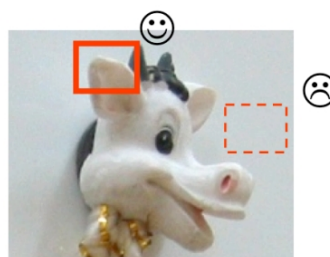
We judge the efficacy of corner or blob detectors on three metrics: repeatability (can we find the same corners under different geometric and photometric transformations), saliency (how “interesting” this corner is), and locality (how small of a region this blob is)



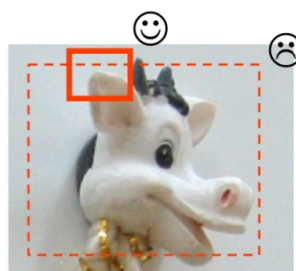
Ideally, we want our corner detector to consistently find the same corners in different images. In this example, we have detected the corner of this cow's ear. We would like our corner detector to be able to detect the same corner regardless of lighting conditions, scale, perspective shifts, and pose variations.



- Saliency



- Locality



La saliency indica quanta “informazione” mi fornisce quella patch

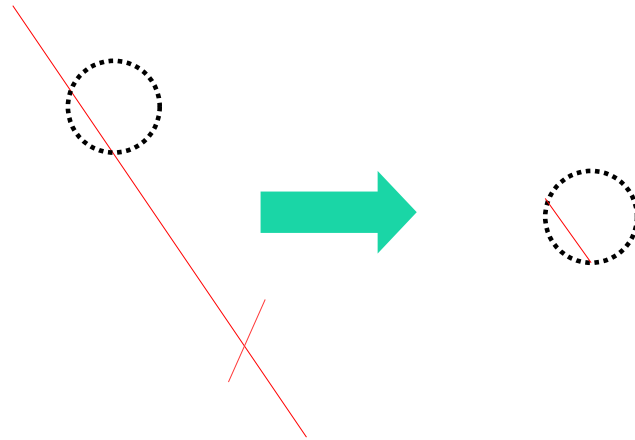
Our corner/blob detector should also pick up on interesting (salient) keypoints. In the top image, an example of an area with poor saliency is indicated by the dashed bounding box. There is not much texture or interesting substance in this area. We also would like our interesting features to have good locality. That is, it should take up a small portion of the whole image. If our blob detector returned the whole image (or nearly the whole image) – e.g., see region indicated by the dashed bounding box– that would defeat the purpose of keypoint detection.

- Textureless patches are nearly impossible to match
- Patches with large contrast changes (gradients) are easier to localize
 - We want edges!
- Are straight edges a good option?
 - Aperture Problem

Dagli esempi precedenti si capisce come per soddisfare le caratteristiche richieste ci vogliano degli edge ovvero delle variazioni di gradiente.

Quindi sicuramente NON aree uniformi o quasi.

E però come devono essere i miei edge?

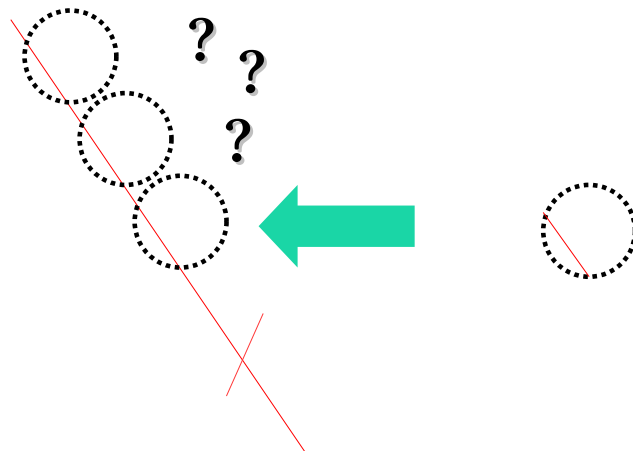


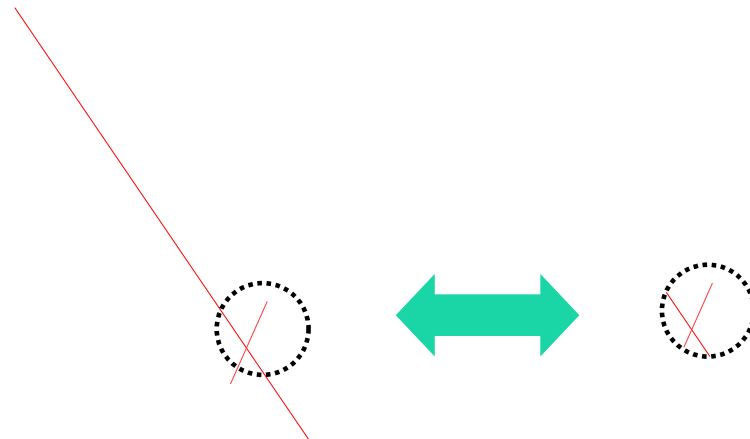
Quindi aree uniformi no, gli edge sono buoni candidati, così come potrebbero essere buoni candidate aree con molta texture
Supponiamo che il circolino sia il mio keypoint

Quando seleziono un keypoint dovrò capire se ci sono corrispondenze nell'altra immagine ovvero se quello stesso keypoint esiste anche dall'altra parte.

Ma una volta estratto, riesco a capire da dove ci può essere corrispondenza?
Se guardo solo la parte interna al cerchio non stimo correttamente il movimento in tutte le direzioni

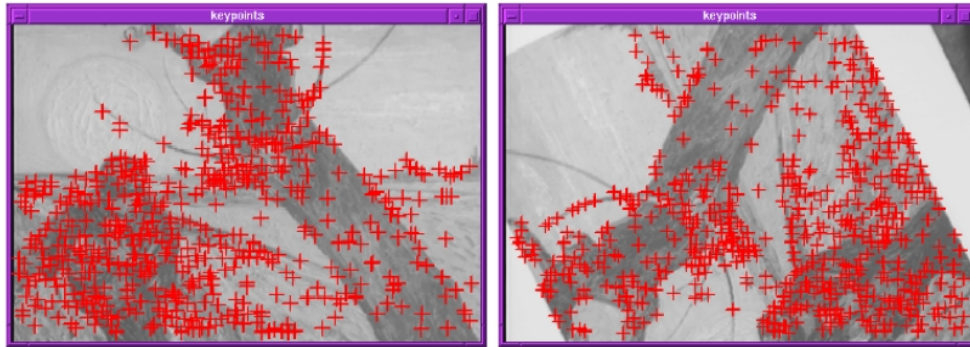
Aperture problem





Se scelgo come feature quella indicata nel circolino, dove ho due edge che hanno un angolo tra di loro. Anche il match è fattibile, riesco senza troppi problemi a capire dove posizionare la mia patch nell'immagine

- In “real” images a corner is easier to match
 - 2 “strong” gradients

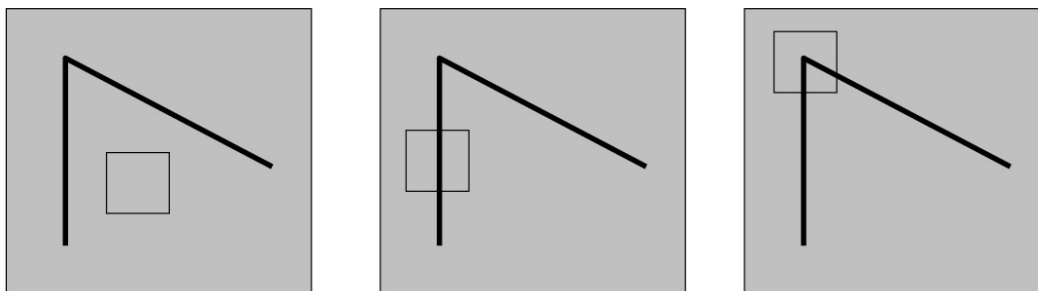


Repeatability – Saliency – Locality

Quindi almeno 2 direzioni dominanti



- What defines whether our patch is a good or bad candidate?



Ok abbiamo capito il trucco, vediamo di formalizzare la cosa e di capire un sistema per estrarre automaticamente i punti “interessanti”

Guardiamo le zone racchiuse dal quadratino. Immaginiamo di dover fare un match. E pensiamo di doverle posizionare.

Regioni “flat” no cambi in nessuna direzione, quindi che la posizioni lì da dove proveniva o leggermente spostata non cambia nulla

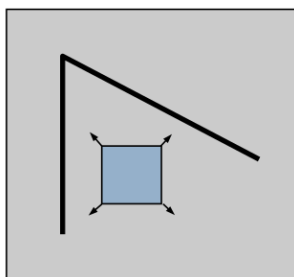
Edge -> la posso posizionare anche spostata (in una sola direzione)

Corner -> qui non si scappa, la posiziono con precisione

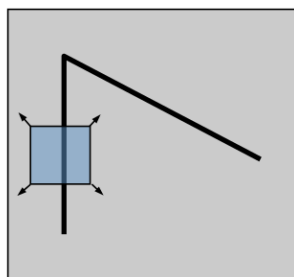
Poi le immagini reali sono un po’ più complesse ma per il momento soprassediamo



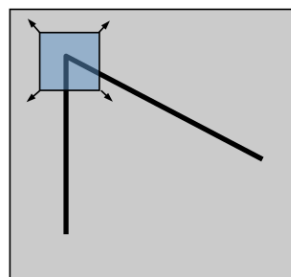
- Let's move a small window (W) on the image
- How the content is changing?



"flat" region:
no change in all
directions



"edge":
no change along the
edge direction



"corner":
significant change in
all directions

Formalizziamo meglio il tutto.

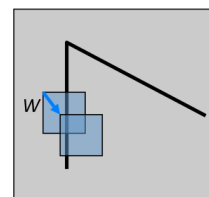
Consideriamo lo stesso rettangolo di prima e immaginiamola come finestra W sull'immagine.

Cosa cambia a muovere la W? Ovvero cambia quanto racchiude quella finestra se la sposto attorno alla posizione originale?



- Let's move window W by (u,v)
 - u and v are small...
- A good estimation can be the Sum of Squared Differences (SSD)
- The SSD error $E(u, v)$ can be defined as

$$E(u, v) = \sum_{(x, y) \in W} [I(x+u, y+v) - I(x, y)]^2$$



Troviamo un qualcosa per misurare precisamente quanto visto la slide precedente. Uso una funzione di errore $E()$. Che mi indica quanto cambia il contenuto di una immagine attorno al punto (x,y) muovendo una finestrella intorno al punto stesso di u,v pixel.

Facile, considero i valori dei pixel della finestra centrata in (x,y) e li confronto con quelli corrispondenti una volta che traslo la finestra di u,v ovvero la centro su $(x+u,y+v)$

Per farlo uso SSD (ma si possono pensare anche altri confronti).

Sembrerebbe oneroso da calcolare...

- Using a Taylor Expansion we can compute $I(x+u, y+v)$

$$I_x = \frac{\partial I(x, y)}{\partial x} \quad I_y = \frac{\partial I(x, y)}{\partial y}$$

$$I(x+u, y+v) = I(x, y) + \frac{I_x u}{1!} + \frac{I_y v}{1!} + \text{higher order terms}$$

$$\approx I(x, y) + I_x u + I_y v$$

$$\approx I(x, y) + \begin{bmatrix} I_x & I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Ma in realtà, per calcolarla possiamo approssimare $I()$ usando Taylor ovvero ricadendo sull'uso del gradiente orizzontale e di quello verticale

- And then replace it in $E(u,v)$ $I_x = \frac{\partial I(x,y)}{\partial x}$ $I_y = \frac{\partial I(x,y)}{\partial y}$

$$\begin{aligned}
 E(u,v) &= \sum_{(x,y) \in W} [I(x+u, y+v) - I(x,y)]^2 \\
 &\approx \sum_{(x,y) \in W} [I(x,y) + I_x u + I_y v - I(x,y)]^2 \\
 &\approx \sum_{(x,y) \in W} [I_x u + I_y v]^2
 \end{aligned}$$

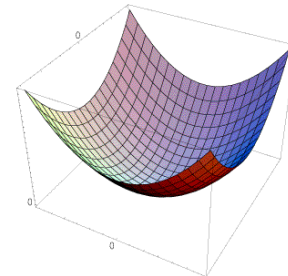
Poi lo sostituisco nella formula e...

- Result

$$I_x = \frac{\partial I(x, y)}{\partial x} \quad I_y = \frac{\partial I(x, y)}{\partial y}$$

$$E(u, v) \approx \sum_{(x, y) \in W} [I_x u + I_y v]^2$$

$$\approx Au^2 + 2Buv + Cv^2$$



$$A = \sum_{(x, y) \in W} I_x^2 \quad B = \sum_{(x, y) \in W} I_x I_y \quad C = \sum_{(x, y) \in W} I_y^2$$

È, geometricamente parlando, una quadrica (un paraboloide ellittico)

- Using a Taylor Expansion $I_x = \frac{\partial I(x, y)}{\partial x}$ $I_y = \frac{\partial I(x, y)}{\partial y}$

$$E(u, v) \approx \sum_{(x, y) \in W} [I_x u + I_y v]^2$$

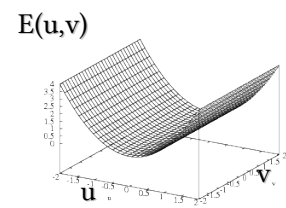
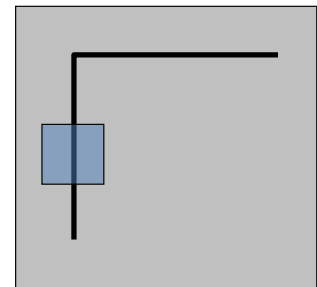
$$\approx Au^2 + 2Buv + Cv^2$$

$$\approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix}$$

- Vertical Edge $\rightarrow I_y=0$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$\approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$



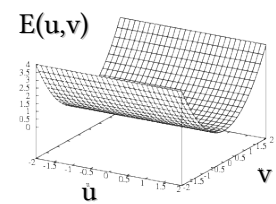
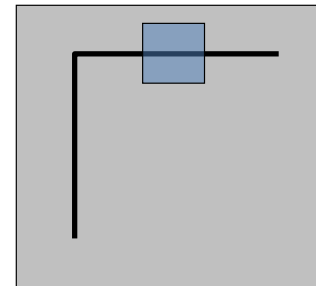
Vediamo come varia nelle varie situazioni

Se non ho edge orizzontali, $E(u, v)$ non dipende da u ...

- Horizontal Edge $\rightarrow I_x=0$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$\approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$



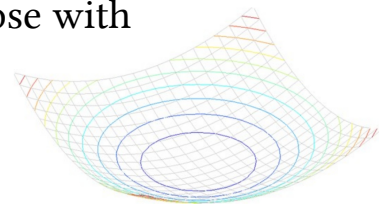
Dualmente, in questo caso, $E(u,v)$ non dipende da v ...

- General Case

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix}$$

- H can be considered as describing an ellipse with

- Axis lenght $\propto$ H eigenvalues
- Axis orientation $\rightarrow$ H eigenvectors



In un caso generale è un parabolode ellittico. Ma ragionevolmente non sarà simmetrico ma più o meno allungato.

Quindi tagliandolo con un piano parallelo agli assi u e v otterrò quasi sempre un'ellissi che avrà un asse maggiore e uno minore

Si può dimostrare che H è legata agli assi di tali ellissi.

In pratica la direzione degli assi è legata agli autovettori e la loro lunghezza è proporzionale agli autovalori

- Given a matrix A eigenvectors x and eigenvalues λ satisfy:
 $Ax = \lambda x$
- Eigenvalues can be found solving
 $\det(A - \lambda I) = 0$
- For a 2×2 matrix like H we can find $\lambda \rightarrow \det \begin{bmatrix} h_{11} - \lambda & h_{12} \\ h_{12} & h_{22} - \lambda \end{bmatrix}$
 - This leads to a **quadratic** equation
 - i.e. λ_1 and λ_2
- Once found λ s, x can be computed as $\rightarrow \begin{bmatrix} h_{11} - \lambda & h_{12} \\ h_{12} & h_{22} - \lambda \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0$

Ci sono infiniti autovalori visto che in $Ax = \lambda x$ posso infilare un qualunque fattore di scala k . Quindi se $\lambda_{1/2}$ sono soluzioni anche $k\lambda_{1/2}$ lo sono. Ergo sistema indeterminato ovvero $\rightarrow$ determinante = 0

L'equazione di secondo grado porta a delle radici

$$\lambda_{1,2} = \frac{1}{2}[h_{11} + h_{22} \pm \sqrt{4h_{12}h_{21} + (h_{11} - h_{22})^2}]$$



$$\lambda_{1,2} = \frac{1}{2} \left[h_{11} + h_{22} \pm \sqrt{4 h_{12} h_{21} + (h_{11} - h_{22})^2} \right]$$

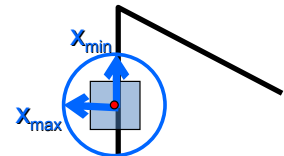
Fattibile ma devo calcolare una radice. Oneroso quindi (ricordiamo che va fatto per ogni punto dell'immagine)

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix}$$

$$Hx_{\max} = \lambda_{\max} x_{\max}$$

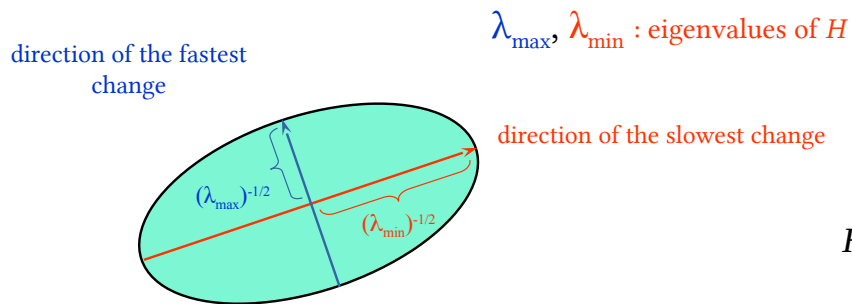
$$Hx_{\min} = \lambda_{\min} x_{\min}$$

- $x_{\max} \rightarrow$ direction of largest increase for $E(u, v)$
- $x_{\min} \rightarrow$ direction of smallest increase for $E(u, v)$
- $\lambda_{\max} \rightarrow$ amount of increase along $x_{\max}$
- $\lambda_{\min} \rightarrow$ amount of increase along $x_{\min}$



Visto che, in generale, mi descrive un'ellisse, avrò un asse minore e uno maggiore cambiamo notazione e da $\lambda_{1,2}$ indichiamoli con max,min

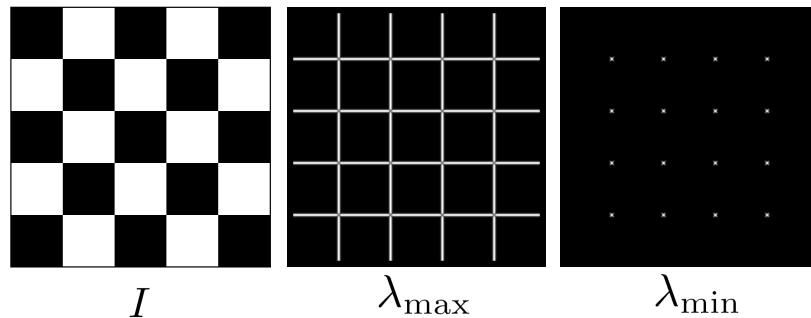
- H can be considered as describing an ellipse with
 - Axis length $\propto$ H eigenvalues
 - Axis orientation $\rightarrow$ H eigenvectors



$$H = R \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R^{-1}$$

La mia H è diagonalizzabile. R è una matrice di rotazione ortonormale e quindi R^T è uguale a R^{-1}

- How $\lambda_{\min}$ and $\lambda_{\max}$ behave on corners?
- $\lambda_{\max}$ is “big” when we have at least one “strong” gradient
- $\lambda_{\min}$ encodes the strenght of the ortogonal gradient



How are $\lambda_{\max}$, $x_{\max}$, $\lambda_{\min}$, and $x_{\min}$ relevant for feature detection?

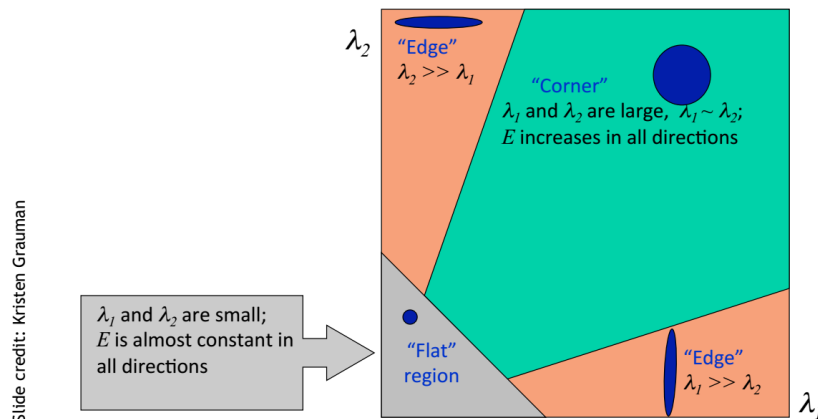
What's our feature scoring function?

Want $E(u,v)$ to be large for small shifts in all directions

the minimum of $E(u,v)$ should be large, over all unit vectors $[u \ v]$

this minimum is given by the smaller eigenvalue ($\lambda_{\min}$) of H

- In a corner both λ_1 and λ_2 are “large”



I valori dei due lambda io li posso usare per capire la forma dell'ellisse

Quando uno dei due è molto più grande dell'altro l'ellisse sarà molto schiacciata
 → 1 edge forte e l'edge ortogonale debole → non ci interessa

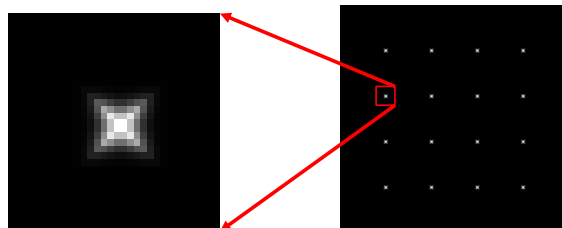
Vogliamo solo punti dove ho due edge forti quindi dove i due lambda sono simili.

Ma anche dove sono sufficientemente grandi. Altrimenti posso avere componenti ortogonali di edge simili ma comunque edge poco marcati → area uniforme

- Summary

- Compute horizontal and vertical gradients
- For each pixel
 - Create H matrix from gradient
 - Compute H eigenvalues
 - Discard pixels with $\lambda_{\min} \ll \lambda_{\max} < \text{threshold}$
- Apply a NMS approach to find pixels where $\lambda_{\min}$ is a local maximum

$$H = \begin{bmatrix} \sum_{(x,y) \in W} I_x^2 & \sum_{(x,y) \in W} I_x I_y \\ \sum_{(x,y) \in W} I_x I_y & \sum_{(x,y) \in W} I_y^2 \end{bmatrix}$$



Questa slide riassume come me li calcolo.

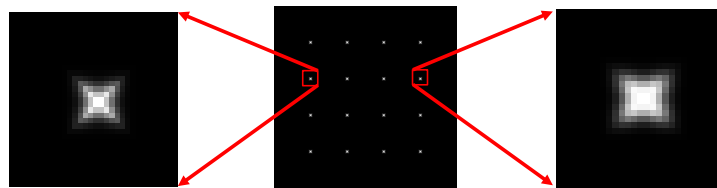
Calcolo su tutta l'immagine I_x e I_y . Poi per ciascun pixel calcolo H e i relativi lambda

Elimino dove lambda piccoli o molto diversi uno dall'altro.

Trovo massimi locali con non maxima suppression

- “Harris” is a little variant of previous detector
 - No need to compute λ s $\rightarrow$ no square roots
 - Weight function $\rightarrow$ based on the distance from the center pixel

$$\theta = \det(H) - \alpha \text{trace}(H)^2$$



“Harris” sta per Chris Harris and Mike Stephens, *A Combined Corner and Edge Detector*, in *Alvey Vision Conference*, vol. 15, 1988.

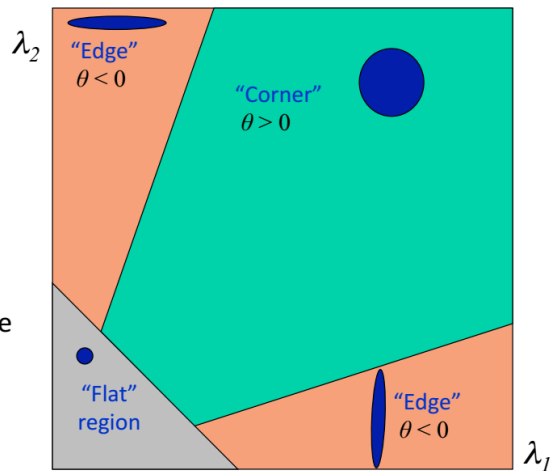
L’idea è semplificare il calcolo dei lambda (che richiede una radice) usando quella formula. Il risultato sarà differente ma si presta ad essere usato con lo stesso scopo.

Nota: $\text{trace}(H)$ = somma elementi in diagonale $h_{11}+h_{22}$

$$\theta = \det(M) - \alpha \text{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

Slide credit: Kristen Grauman

- Fast approximation
 - Avoid computing the eigenvalues
 - α : constant (0.04 to 0.06)

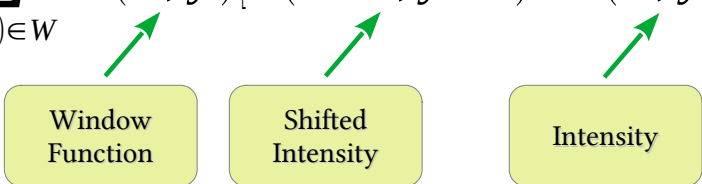


In questo caso cambia come valuto il teta

- Shi et al. (1994) $\lambda_{min}^{\frac{1}{2}} \rightarrow$
- Harris et al. (1998) $\rightarrow \theta = \det(H) - \alpha \text{trace}(H)^2 = \lambda_0 \lambda_1 - \alpha (\lambda_0 + \lambda_1)^2$
- Triggs et al. (2004) $\rightarrow \lambda_{min} - \alpha \lambda_{max}$
- Brown et al. (2005) $\rightarrow \lambda_{min} \approx \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2} = \frac{\det(H)}{\text{trace}(H)}$

Poi nel tempo ne sono stati proposti altri...

- Harris also introduced a window function
 - The idea is to weight distance

$$E(u, v) = \sum_{(x, y) \in W} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$


Window Function

Shifted Intensity

Intensity

Discorso già visto. Noi usiamo tutti i pixel di una finestra W , ma ha senso che il loro contributo sia lo stesso indipendentemente dalla distanza dal centro della patch?

Usiamo una funzione che mi pesa il tutto.



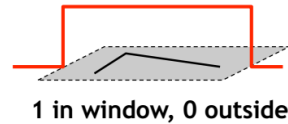
$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- Option 1: uniform window

– Sum over square window

$$M = \sum_{x,y} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

– Problem: not rotation invariant

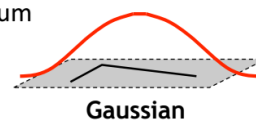


- Option 2: Smooth with Gaussian

– Gaussian already performs weighted sum

$$M = g(\sigma) * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

– Result is rotation invariant



Slide credit: Bastian Leibe

Di solito si usa una gaussiana

- Summary

- Compute horizontal and vertical gradients

- Apply window function

- For each pixel

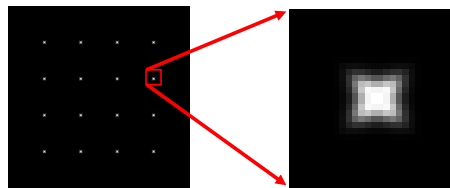
- Create H matrix from gradients

- Compute θ

- Discard pixels with $\theta < 0$ or anyway below a given threshold

- Apply a NMS approach to find pixels where $\lambda_{\min}$ is a local maximum

$$H(\sigma_I, \sigma_D) = g(\sigma_I) \begin{bmatrix} \sum_{(x,y) \in W} I_x^2(\sigma_D) & \sum_{(x,y) \in W} I_x I_y(\sigma_D) \\ \sum_{(x,y) \in W} I_x I_y(\sigma_D) & \sum_{(x,y) \in W} I_y^2(\sigma_D) \end{bmatrix}$$



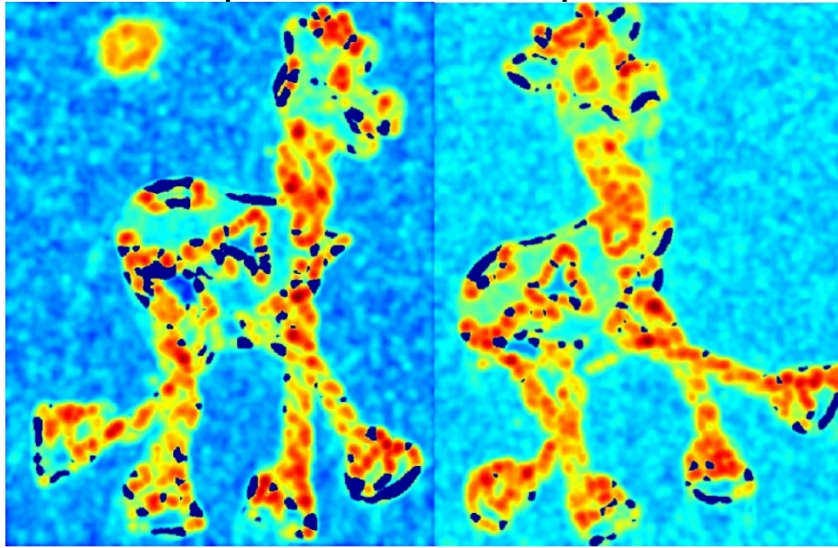
Di fatto sfrutto la proprietà commutativa delle varie convoluzioni e applico prima una gaussiana all'immagine e dopo calcolo i gradienti.

Harris Corner Detector steps



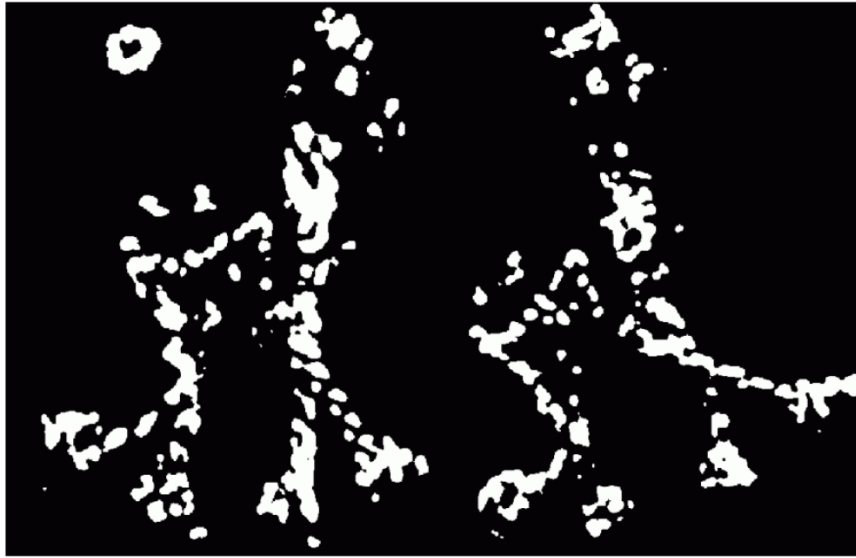
Slide adapted from Darya Frolova, Denis Simakov

Harris Corner Detector steps



Slide adapted from Darya Frolova, Denis Simakov

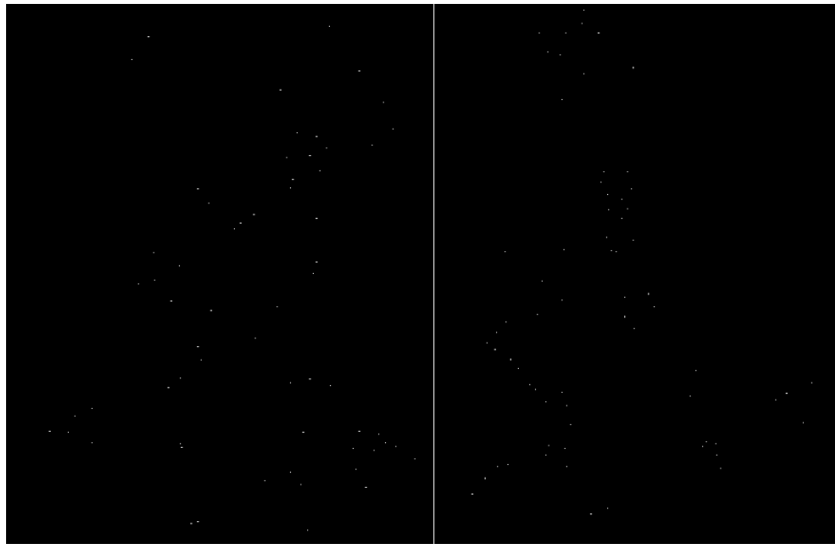
Rosso → risposta alta



Prima dell'NMS

Harris Corner Detector steps

UNIVERSITÀ
DI PARMA



Slide adapted from Darya Frolova, Denis Simakov

NMS applicato

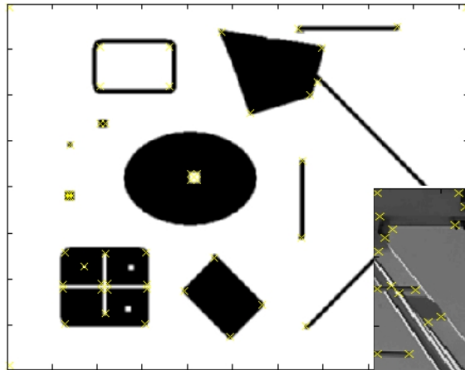
Harris Corner Detector steps



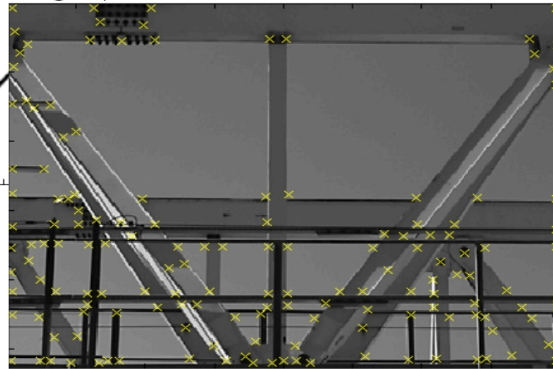
Slide adapted from Darva Frolova, Denis Simakov

Il fatto di vedere il risultato su due immagini è molto interessante.

Infatti, riportando sulle immagini posso vedere come, in buona parte dei casi, la feature estratta da una delle immagini la ritrovi pari pari nell'altra.



Effect: A very precise
corner detector.

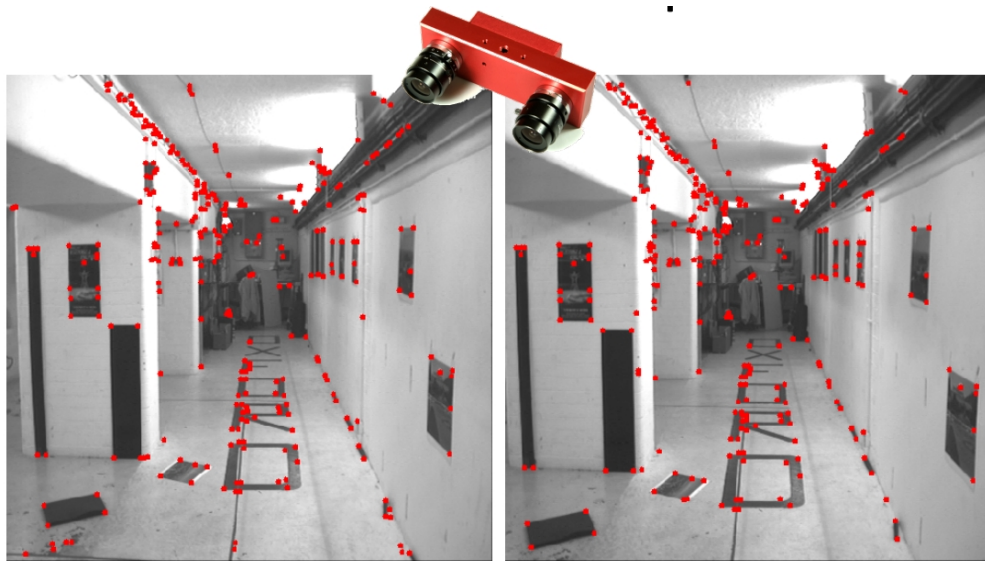


Slide credit: Krystian Mikolajczyk

Altro esempio



Slide credit: Krystian Mikolajczyk



- How much keypoints extraction is robust to image transformations?

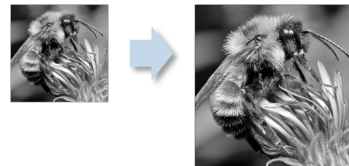
- Rotation (traslation)



- Intensity change



- Scale

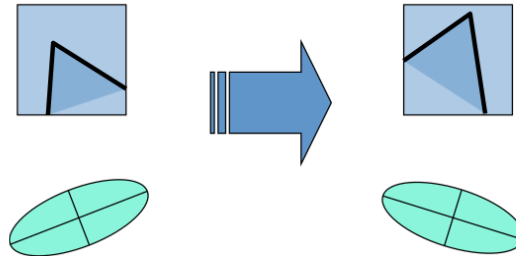


Va beh ma questa tecnica risponde bene ai requisiti che desideravamo? Ad esempio trovo gli stessi keypoint se ruoto l'immagine, cambio intensità o scalo?



- Ellipse rotates but eigenvalues are the same

$$H = R \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R^{-1}$$

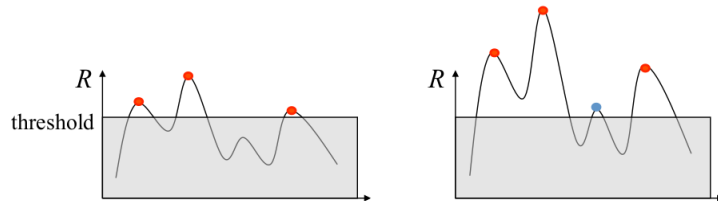


Invariant to Rotation

La rotazione non mi fa un baffo!



- Intensity change can be modeled as $g(r, c) = a \cdot f(r, c) + b$
- Derivative operations make result invariant wrt b
- Not so true about a ...



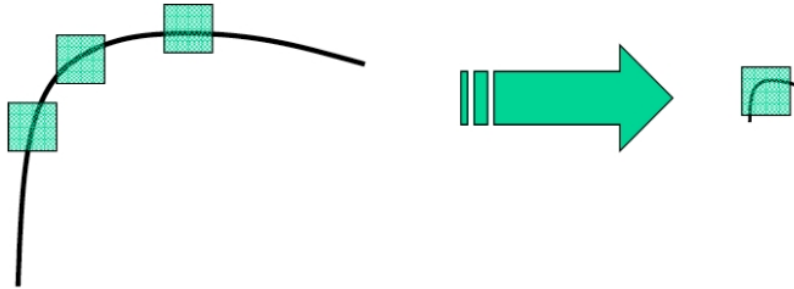
Partially Invariant to Intensity Changes

Qui viceversa in parte. È vero che derivo e quindi il $+b$ sparisce ma l' a mi può far emergere dei picchi che prima non avevo

Nota: non soglio l'immagine ma ad un certo punto soglio quel $teta$... e l'effetto è simile



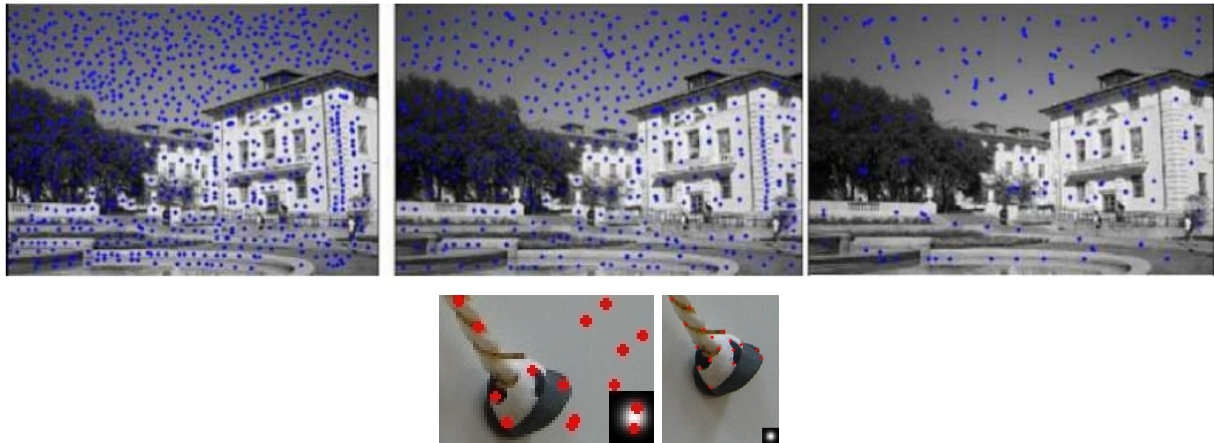
- Right $\rightarrow$ corner
- Left $\rightarrow$ corner



Not Invariant to Scale Changes

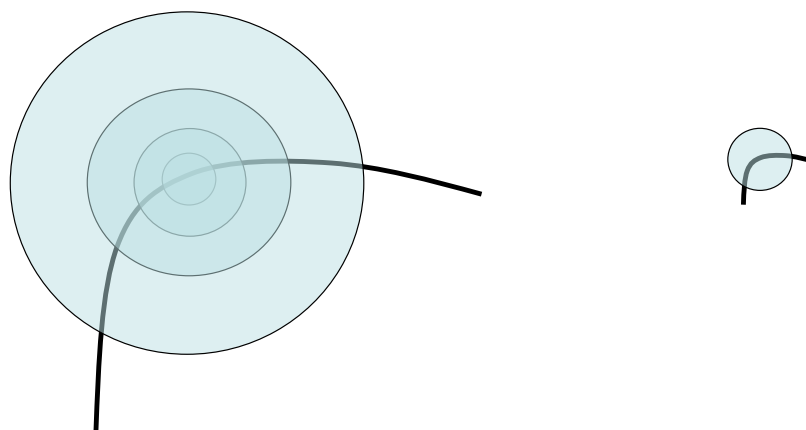
La scala mi frega completamente

- The keypoints are highly dependent on window size



Similmente se cambio le dimensioni della W ottengo risultati differenti
La zampa del bestio è invece lei a scale differenti, non la finestra

- Consider circles of different size around a point
- We can find a size that look similar in both images

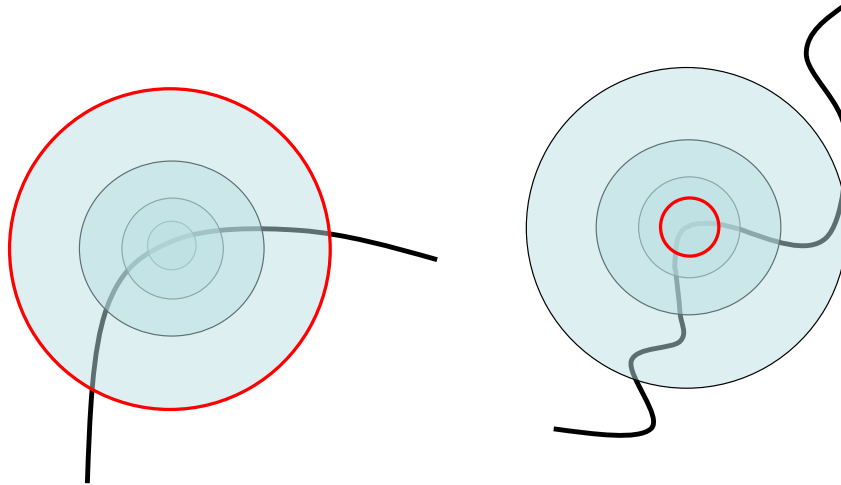


Esiste però la possibilità di affrontare il problema. Di fatto è stato formalizzato in SIFT (1999)

Supponiamo di usare W di dimensioni differenti e visto che usiamo la funzione di peso immaginiamo le mie finestre come cerchi



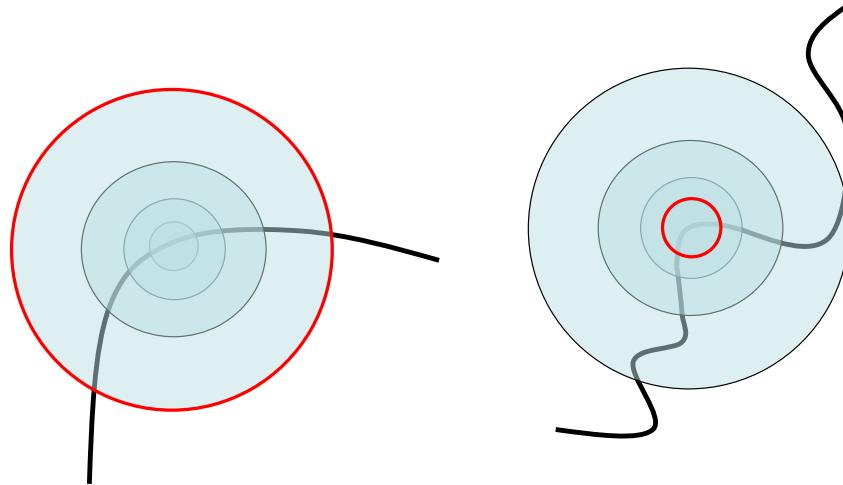
- How do we select appropriate circles size in both images?



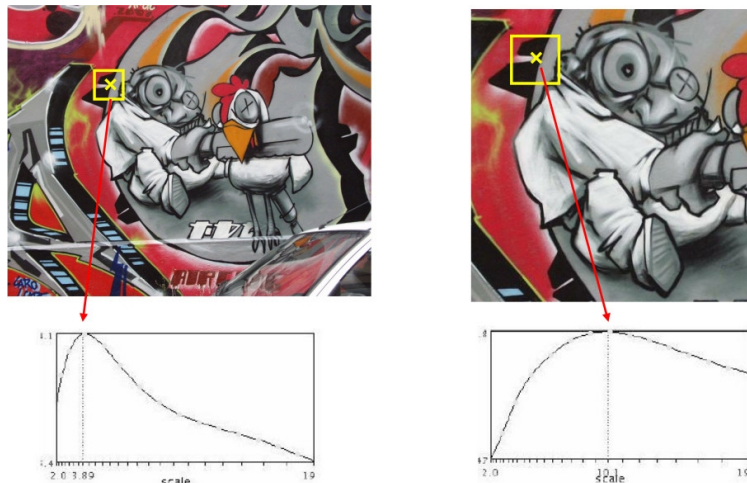
All'atto pratico io provo tante finestre e prendo la finestra in cui ho il massimo del parametro utilizzato per valutare la "bontà" di quel punto (sempre che il massimo sia comunque accettabile)



- We can select the window size that **maximize** Harris operator



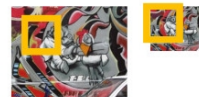
- Example



Nei grafici si vede come varia il mio operatore (qui non è esattamente Harris ma sorvoliamo) al variare delle dimensioni della patch centrata in quel punto.

Nelle immagini riporto la dimensione della patch. Come si può vedere e come è ragionevole aspettarsi nell'immagine ingrandita anche la patch è più grande per lo stesso punto.

- Instead computing θ for larger and larger windows a pyramidal approach can be used



(sometimes need to create in-between levels, e.g. a $\frac{3}{4}$ -size image)

Bello, quindi tutto il riassunto dei passi visto un po' di slide prima lo ripeto per finestre di dimensioni differenti?

Si può fare ma sarebbe poco efficiente. Patch grandi $\rightarrow$ tanti punti da considerare

Si usa viceversa una patch di dimensione fissa e piccolina e la si applica alla stessa immagine scalata

Approccio piramidale: di solito genero tante immagini dividendo le dimensioni per radice di 2 fino a un minimo predefinito



- **HARRIS-LAPLACIAN**
 - K. Mikolajczyk, C. Schmid, Indexing based on scale invariant interest points, ICCV 2001
- **SIFT**: Scale Invariant Feature Transform
 - D. Lowe. "Distinctive Image Features from Scale-Invariant Keypoints", IJCV 2004
- **FAST**: Features from Accelerated Segment Test
 - Edward Rosten and Tom Drummond, "Machine learning for high speed corner detection" in 9th European Conference on Computer Vision, vol. 1, 2006
- **SURF**: Speeded-Up Robust Features
 - Herbert Bay, Andreas Ess, Tinne Tuytelaars, Luc Van Gool, "SURF: Speeded Up Robust Features", Computer Vision and Image Understanding (CVIU), 2008
- **MSER**: Maximally Stable Extremal Regions
 - J. Matas, O. Chum, M. Urban, and T. Pajdla. "Robust wide baseline stereo from maximally stable extremal regions." Proc. of British Machine Vision Conference, 2002.
- **BRISK**: Binary Robust Invariant Scalable Keypoint
 - Stefan Leutenegger ; Margarita Chli ; Roland Y. Siegwart, 2011 International Conference on Computer Vision

